

2024/2025

Memoria Práctica Final



Elena García Villarabide

Análisis de Datos Clínicos

Universidad de Alicante

2024/2025

Índice

1. Introducción.....	3
2. NIVEL BÁSICO	3
2.1. Análisis exploratorio de los datos	3
2.1.1. Exploración de las características	3
2.1.2. Visualización de los datos.....	5
2.1.3. Cálculo del desequilibrio de clases	8
2.1.4. Eliminación de las variables sobrantes.....	9
2.2. Clasificadores básicos	11
2.2.1. Evaluación de un modelo básico – DummyClassifier.....	11
2.2.2. LogisticRegression	12
2.2.3. RandomForestClassifier	14
2.2.4. Comparación de los clasificadores básicos	15
2.3. Ajuste de los parámetros	16
2.3.1. Ajuste para LogisticRegression	16
2.3.2. Ajuste para RandomForestClassifier	18
2.3.3. Comparación de los clasificadores básicos con hiperparámetros	20
2.4. Balanceo de clases	21
2.4.1. SMOTE (oversampling)	21
2.4.2. RandomUnderSampler (undersampling)	27
2.4.3. Comparación de los clasificadores básicos tras el balanceo	32
2.5. Test de contraste de hipótesis	33
3. NIVEL MEDIO	35
3.1. Algoritmos de selección automática de atributos relevantes.....	35
3.1.1. SelectKBest.....	35
3.1.2. Recursive Feature Elimination (RFE)	37
3.2. Clasificadores medios.....	40
3.2.1. LogisticRegression	41
3.2.2. RandomForestClassifier	42
3.2.3. DecisionTreeClassifier	44
3.2.4. KNeighborsClassifier	46
3.2.5. GradientBoostingClassifier	48
3.2.6. XGBoost	50
3.2.7. Naive Bayes.....	52
3.3. Ajuste de los parámetros	55
3.3.1. LogisticRegression	55

3.3.2.	RandomForestClassifier	56
3.3.3.	DecisionTreeClassifier	57
3.3.4.	KNeighborsClassifier	57
3.3.5.	GradientBoostingClassifier	58
3.3.6.	XGBoost	59
3.3.7.	Naive Bayes	60
3.3.8.	Comparación de los clasificadores medios con hiperparámetros.....	61
3.4.	Clasificadores sensibles al coste	62
3.4.1.	LogisticRegression	62
3.4.2.	RandomForestClassifier	65
3.4.3.	DecisionTreeClassifier	67
3.4.4.	Comparación de clasificadores medios tras class_weight	69
3.5.	Tests estadísticos por pares	70
3.6.	Cálculos adicionales	74
4.	Referencias	76
5.	Bibliografía	78

ESTUDIO SOBRE INDICADORES DE SALUD PARA LA DIABETES

1. Introducción

El objetivo principal de esta práctica es identificar los factores más relevantes para predecir de manera correcta si una persona tiene diabetes o si es saludable. Para ello se deben aplicar técnicas de machine learning sobre un conjunto de datos reales acerca de indicadores de salud relacionados con la diabetes.

El desarrollo se ha llevado a cabo en un Google Colab utilizando Python y diferentes librerías estudiadas previamente en clase. Por otro lado, la base de datos sobre la que se trabaja fue descargada desde la plataforma Kaggle e incluye información detallada de más de 250.000 personas.

Antes de comenzar con la práctica, conviene saber qué es la diabetes. Según el ministerio de sanidad español, la diabetes mellitus es una enfermedad producida por una alteración del metabolismo, caracterizada por un aumento de la cantidad de glucosa en la sangre, lo que conlleva a la aparición de complicaciones microvasculares y cardiovasculares. La diabetes es una enfermedad bastante común, afectando a entre el 5 y el 10% de la población general y su detección precoz puede ser clave.

2. NIVEL BÁSICO

2.1. Análisis exploratorio de los datos

2.1.1. Exploración de las características

Para poder cargar la base de datos primero hay que instalar un paquete de Python llamado ucimlrepo que permite acceder a datasets del repositorio UCI Machine Learning Repository desde el código.

```
[ ] pip install ucimlrepo
```

Para comenzar, se importan las librerías necesarias como pandas para manejar los datos y fetch_ucirepo para descargar la base de datos del repositorio. A continuación, se descarga el dataset con ID 891 que es CDC Diabetes Health Indicators Dataset, un dataset con indicadores de salud de miles de personas.

Una vez cargado el dataset en Google Colab, se puede proceder a la separación de dichos datos en dos tablas diferentes, una de variables explicativas llamada 'X' y otra tabla 'y' que contiene únicamente la variable objetivo. Además de ordenar nuestras variables, con este paso estamos omitiendo la variable ID ya que no aporta valor predictivo para el estudio.

Por último, se muestran las primeras filas de ambas tablas con el comando .head(). Esto permite tener una visión general de cómo están estructurados los datos y así poder planificar el siguiente paso del análisis exploratorio.

```

from ucimlrepo import fetch_ucirepo

# fetch dataset
cdc_diabetes_health_indicators = fetch_ucirepo(id=891)

# data (as pandas dataframes)
X = cdc_diabetes_health_indicators.data.features
y = cdc_diabetes_health_indicators.data.targets

X.head()
y.head()

```

Para comenzar el análisis y poder sacar su máximo potencial, es importante familiarizarse con el conjunto de datos con el que se va a trabajar. Por ello, en esta sección se va a analizar el **tipo de cada variable** (numérica o categórica) y se va a comprobar si hay **valores faltantes**.

```

# 1. Ver los tipos de datos de cada columna
print("\nTipos de datos:")
print(X.dtypes)
print(y.dtypes)

# 2. Ver si hay valores nulos
print("\nValores nulos por columna:")
print(X.isnull().sum())
print(y.isnull().sum())

```

Como salida del código anterior se obtiene que todas las variables son int64 lo que significa que son números enteros, es decir, sin decimales. A su vez, se indica que no existen valores nulos en ninguna variable de la base de datos.

No obstante, esto no garantiza que todos los valores sean correctos. Algunas variables son binarias por lo que si contienen un valor diferente de 0 o 1 no se podrían analizar correctamente. A su vez, hay variables que solo permiten un rango específico de valores. Por ejemplo, la variable GenHlth debería tomar valores dentro del rango de 1 a 5, por lo que si en la base de datos hubiera un valor fuera de ese rango se consideraría inválido. Estos rangos se pueden consultar en el enlace proporcionado en la práctica donde explica cada variable mediante una breve descripción.
<https://archive.ics.uci.edu/dataset/891/cdc+diabetes+health+indicators>

Para asegurarnos que las variables binarias solo tienen valores de 0 o 1 se define una lista que contiene todos los nombres de las columnas en X que son binarias y se recorre cada una de ellas. Para cada columna se recorre el conjunto de valores únicos que tiene y luego se confirma que están dentro del conjunto {0,1}. Si no es así, se imprime un mensaje indicando la variable con valores no esperados y cuáles son esos valores. Se hace el mismo procedimiento con la tabla y.

Tras ejecutar el código no se muestra ningún mensaje en la salida, lo que quiere decir que todos los valores son los esperados.

Se debe realizar el mismo procedimiento para todas las variables que tienen un rango específico de valores. Para ello se crea un diccionario que asocia cada variable con su rango permitido. Después se recorre cada entrada del diccionario con un bucle for. Si se encuentran valores fuera del rango esperado, se imprimirá un mensaje indicando el nombre de la columna y otro proporcionando información sobre esos valores.

Se ha ejecutado el código para comprobar los valores fuera de rango y no se muestra ningún mensaje de advertencia, por lo que todos los valores están dentro de rango.

Con estas dos comprobaciones se han analizado casi todas las variables. Aún queda una que no es binaria ni viene indicada por un rango exacto. Esta variable es el BMI. El BMI es una medida que relaciona el peso y estatura de una persona. Sus valores pueden variar dentro de diferentes rangos. Si es menor de 18,5 significa que la persona se encuentra en peso insuficiente y si es mayor de 30 se encuentra dentro del rango de obesidad. Hay casos extremos de BMIs superiores a 100 pero son extremadamente escasos. Para esta práctica he definido un rango lógico para BMI entre 10 y 60. Se ha comprobado los valores como en las variables anteriores y ha salido el siguiente mensaje tras ejecutar el código:

```
Valores de BMI fuera del rango razonable: [63 61 74 62 64 66 73 85 67 65 70 82 79 92 68
72 88 96 81 71 75 77 69 76 87 89 84 95 98 91 86 83 80 90 78]
```

Aunque se han identificado algunos valores sumamente altos, como 85, 90 o incluso superiores, se ha considerado que pueden corresponder a casos reales de personas con obesidad mórbida. Por ello, se ha decidido no eliminar estos datos del análisis. Solo se eliminarían los casos completamente imposibles como un BMI de 2 o de 300 porque se podría confirmar que serían valores erróneos.

2.1.2. Visualización de los datos

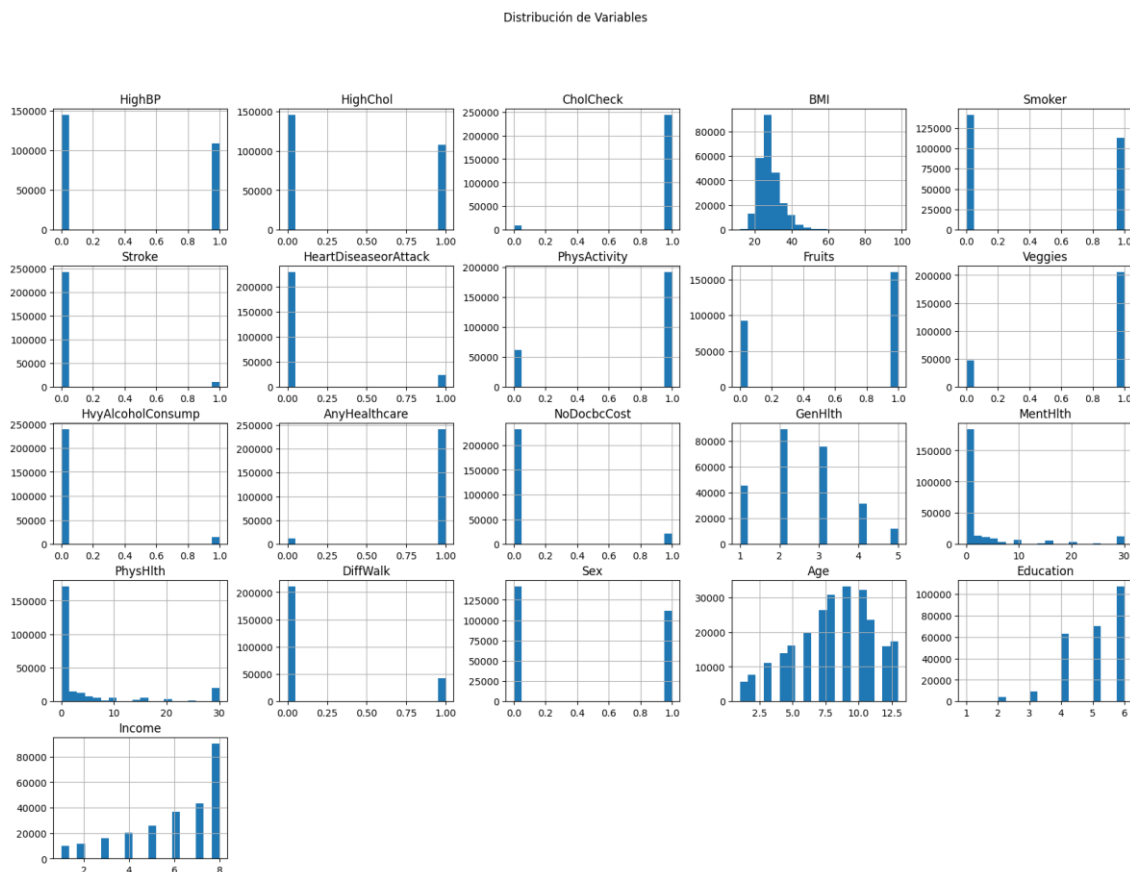
Una vez se han comprobado todos los datos de las variables, se puede pasar al siguiente paso del estudio, la visualización de los datos. En esta parte se utilizan gráficos para entender la distribución de las variables y la relación entre ellas.

Una manera sencilla de ver todos los datos es generando **histogramas** para todas las variables de conjunto de datos de las variables explicativas, también denominado X (Ref 1).

```
import seaborn as sns
import matplotlib.pyplot as plt

# Distribución de las variables
X.hist(figsize=(20, 14), bins=20)
plt.suptitle("Distribución de Variables")
plt.show()
```

El siguiente paso es interpretar los gráficos que se han generado:



- Variables binarias: en las variables que solo toman valores de 0 o 1, los histogramas muestran 2 barras. Si una de las barras es mucho más alta que la otra significa que hay desequilibrio entre los valores. Un claro ejemplo de desequilibrio es 'Veggies' en donde la barra con valor igual a 1 es cuatro veces mayor que la barra con valor 0. Esto quiere decir que hay cuatro veces más personas que afirman comer una o más verduras al día.
- Variables categóricas: en estas variables se pueden observar las categorías que son más frecuentes. Por ejemplo, 'GenHlth' almacena las respuestas a la pregunta de cómo describirías tu salud en general mediante una escala del 1 al 5. Al analizar el histograma se puede observar que la mayoría de la gente marcó respuestas en el centro de la escala como 'muy buena' o 'buena'.
- Variables continuas: la variable continua más clara en este estudio es 'BMI' pero también contamos con las de salud tanto física como mental. En el BMI se puede observar un claro pico entre los valores 25 y 30, lo que significa que muchos pacientes están dentro del rango de sobrepeso. Con respecto a la salud mental y física, hay un pico destacable en 0, indicando que casi todos los pacientes no han tenido problemas ni mentales ni físicos en los últimos 30 días.

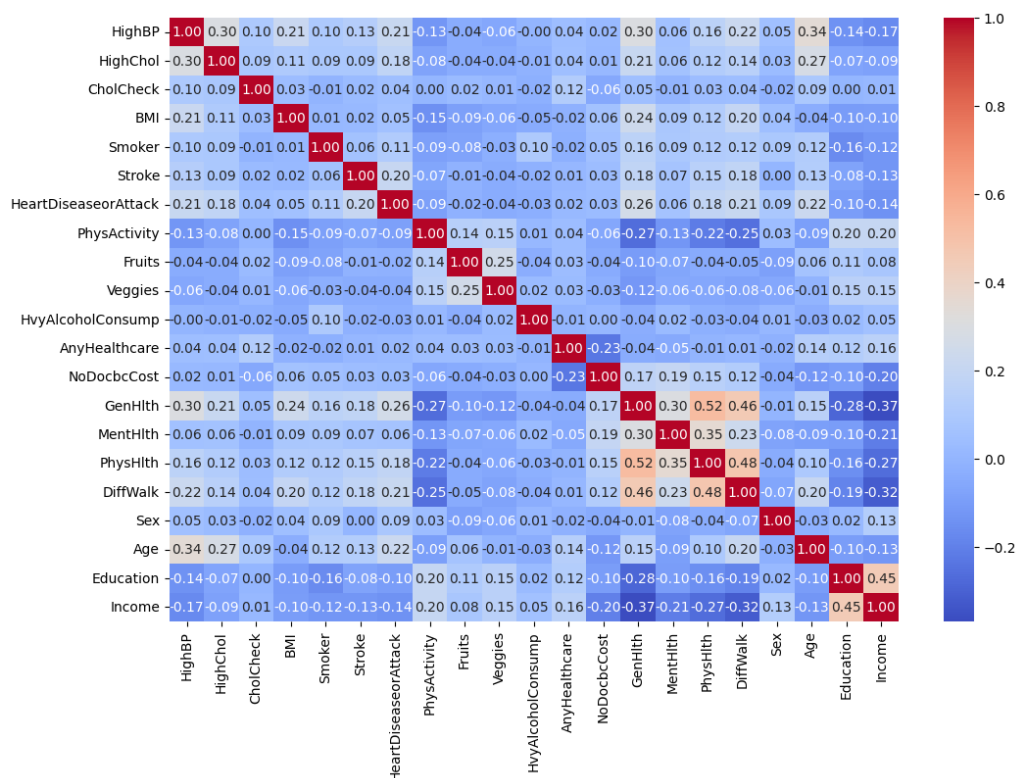
Para analizar si las variables tienen relación entre ellas se emplea la **matriz de correlación** (Ref 2). Ésta es una herramienta muy útil en el análisis exploratorio ya que muestra el coeficiente de correlación entre dos variables con valores que van desde el -1 al 1. Cuanto más cercano sea el valor a 1, más fuerte y positiva es la correlación, es decir, si una de las variables aumenta, la otra también lo haría. Por el contrario, si el valor es cercano a -1 indica una correlación negativa fuerte o, en otras palabras, si una variable aumenta, la otra disminuye. Otra posibilidad a tener en cuenta es que el valor sea cercano a 0, lo que significa que no hay relación lineal clara entre las dos variables a estudiar.

Al calcular la matriz se pueden detectar posibles relaciones entre variables, lo que puede ayudar a entender los datos o a eliminar variables que aporten la misma información al modelo, lo que permitiría eliminar una de ellas.

```
[ ] import seaborn as sns
import matplotlib.pyplot as plt

# Calcula la matriz de correlación
corr_matrix = X.corr()

# Muestra la matriz de correlación
plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.show()
```



En la matriz de correlación se puede observar que la diagonal esta compuesta por la unidad. Esto es lo habitual ya que cada variable está perfectamente correlacionada consigo misma. Además, se pueden detectar algunas correlaciones moderadas que aparecen en la matriz en un color anaranjado.

- 'Education' e 'Income' muestran una correlación de 0,45, lo cual tiene sentido ya que mayores niveles educativos suelen estar asociados a mayores ingresos.
- Las variables 'GenHlth', 'PhysHlth' y 'DiffWalk' están moderadamente relacionadas entre sí, algo esperable ya que peor salud física implica también peor salud general y suelen asociarse con mayores dificultades de movilidad.

2.1.3. Cálculo del desequilibrio de clases

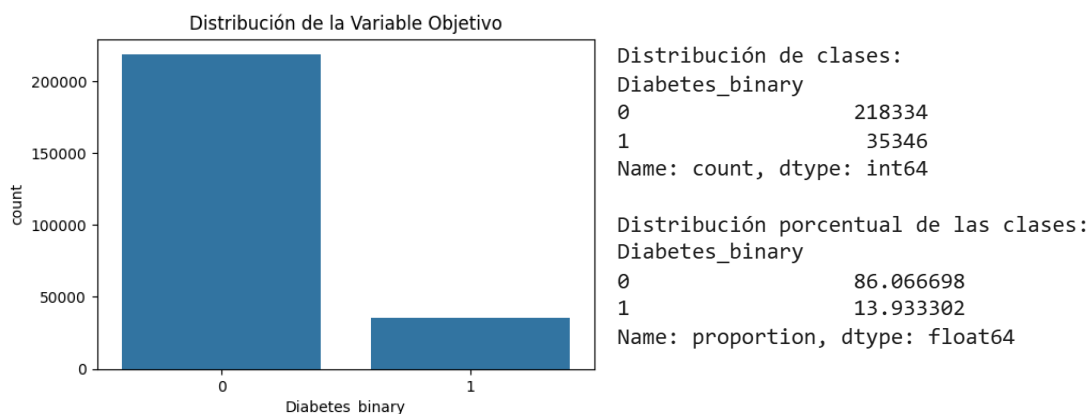
Uno de los pasos fundamentales en el análisis exploratorio es calcular el desequilibrio de clases de la variable objetivo. Este paso consiste en contar cuántas instancias hay en cada categoría de la variable 'Diabetes_binary'. Este análisis permite detectar si el conjunto de datos está desbalanceado, algo que puede influir negativamente en la fiabilidad de los modelos de clasificación porque tienden a predecir con mayor precisión la clase mayoritaria.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Visualización de la distribución de la variable objetivo
plt.figure(figsize=(6, 4))
sns.countplot(x=y["Diabetes_binary"])
plt.title("Distribución de la Variable Objetivo")
plt.show()

# Distribución de clases en números absolutos
print("Distribución de clases:")
print(y.value_counts())

# Distribución porcentual
print("\nDistribución porcentual de las clases:")
print(y.value_counts(normalize=True) * 100)
```



Una vez analizada 'Diabetes_binary' se observa un fuerte desequilibrio de clases, con aproximadamente un 86% de personas sin diabetes (clase 0) y solo un 14% con diabetes (clase 1).

Este desbalance seguramente vaya a afectar negativamente al rendimiento de los modelos de clasificación ya que éstos tienden a favorecer a la clase mayoritaria, lo que puede acabar en una alta precisión, pero bajo rendimiento para detectar correctamente casos de diabetes, es decir, de la clase minoritaria en este caso. Esto se debe a que, si un modelo asume que todas las personas son sanas, ya contaría con una tasa de aciertos del 86%. Por ello, será importante tener el desequilibrio de clases en cuenta al evaluar los resultados.

2.1.4. Eliminación de las variables sobrantes

Antes de continuar con el estudio y tras haber explorado y visualizado la base de datos proporcionada, se debe llevar a cabo uno de los pasos más determinantes del proyecto: analizar todas y cada una de las variables de la tabla de variables explicativas y comprobar su relación con la diabetes. De este modo, si alguna variable que no se relaciona con la enfermedad, podría ser descartada del estudio, lo que permitiría ahorrar tiempo de procesamiento de datos y reducir ruido que pudiera afectar a los resultados finales.

Para realizar una correcta selección de las variables se ha de revisar de manera crítica todas las variables explicativas con la finalidad de eliminar las que no aportan información relevante para predecir la diabetes.

Variables conservadas

Se han conservado las siguientes variables explicativas por su relación directa o indirecta con la diabetes:

- **‘HighBP’**: la hipertensión arterial tiene una fuerte relación con la diabetes y se afectan mutuamente. Según un estudio realizado por Adeslas, *“La mitad de las personas con diabetes padecen hipertensión arterial.”* Hay numerosos estudios que incluyen la hipertensión como factor de riesgo en personas con diabetes.
- **‘HighChol’**: la diabetes tiende a reducir los niveles de colesterol de alta densidad o también conocido como colesterol bueno, y a aumentar los niveles de triglicéridos y colesterol de baja densidad o malo. Por ello, el colesterol se asocia con la diabetes.
- **‘BMI’**: el índice de masa corporal es la medida por excelencia empleada como indicador del sobrepeso o la obesidad, dos factores de riesgo de desarrollar diabetes tipo 2.
- **‘Smoker’**: diversos estudios que han demostrado que existe una relación directamente proporcional entre el número de cigarrillos que fuma una persona y su riesgo de desarrollar diabetes.
- **‘Stroke’** y **‘HeartDiseaseorAttack’**: las enfermedades cardiovasculares y la diabetes están fuertemente relacionadas. Es mucho más probable que una persona con diabetes desarrolle una enfermedad cardíaca.
- **‘PhysActivity’**: la actividad física es un factor importante en la prevención de la diabetes ya que ayuda a controlar los niveles de azúcar en sangre y a mantener un peso saludable. Es un factor protector.

- **‘Fruits’ y ‘Veggies’**: llevar una dieta rica en frutas y verduras ha demostrado reducir el riesgo de desarrollar diabetes, incluso en un 60%.
- **‘HvyAlcoholConsump’**: el consumo excesivo de bebidas alcohólicas puede aumentar el riesgo de diabetes ya que conduce a niveles de azúcar en sangre menos predecibles y afecta negativamente a la regulación de la glucosa.
- **‘DiffWalk’**: hay varios motivos por los que una persona puede tener dificultades para andar. Un nivel alto de glucosa puede provocar daños en los nervios de las extremidades inferiores, lo que puede afectar a la circulación sanguínea y causar dolor o dificultades para andar. Esto se denomina neuropatía diabética.
- **‘Sex’ y ‘Age’**: son los dos factores demográficos más importantes. Se ha confirmado que la diabetes de tipo 2 es más común en personas mayores de 45 años y afecta a los hombres y mujeres en proporciones diferentes.

Variables eliminadas

Estas variables se han eliminado ya que no se han considerado relevantes para el estudio sobre indicadores de la diabetes por los siguientes motivos:

- **‘CholCheck’**: esta variable indica si la persona se ha medido el colesterol, pero, en caso haber realizado el análisis, no proporciona información clínica por lo que no aporta información sobre su salud.
- **‘AnyHealthcare’ y ‘NoDocbcCost’**: ambas se relacionan con la posibilidad de acceso a un médico o servicio sanitario, pero no aporta información clínica.
- **‘GenHlth’, ‘MentHlth’ y ‘PhysHlth’**: estas tres variables son subjetivas y provenientes de una respuesta de la persona a estudiar por lo que pueden estar sesgadas por percepción personal. Pueden aportar información general sobre la persona, pero en esta práctica no se van a considerar como indicador de diabetes y nos vamos a centrar en otros datos más objetivos.
- **‘Education’ e ‘Income’**: ambos indicadores se podrían suponer que tienen relación con los hábitos de salud de una persona, pero no aportan valor predictivo suficiente en este estudio.

Una vez analizadas las variables y seleccionadas las que se van a tener en cuenta de ahora en adelante, se procede a la eliminación de las variables no seleccionadas de la tabla X.

```
[ ] X = X.drop(columns=[
    "CholCheck", "AnyHealthcare", "NoDocbcCost",
    "GenHlth", "MentHlth", "PhysHlth", "Education", "Income"
])
X.head()
```

2.2. Clasificadores básicos

2.2.1. Evaluación de un modelo básico – DummyClassifier

Para establecer un punto de partida en la evaluación del modelo, se implementó un DummyClassifier con la estrategia "most_frequent" (Ref 3), lo cual significa que el modelo siempre predice la clase mayoritaria (en este caso, la clase 0 son las personas sin diabetes). Este tipo de modelo no utiliza ninguna información de las variables predictoras, por lo que sirve como modelo base para comparar con modelos más complejos posteriormente.

Se utilizó validación cruzada estratificada (StratifiedKFold) con 10 particiones (Ref 4), lo que asegura que en cada partición se mantenga la proporción original de clases. Se ha realizado de esta forma ya que la base de datos puede ocasionar problemas en los clasificadores debido a su excesivo desequilibrio de clases. Durante la evaluación, se calcularon las siguientes métricas:

- **Matriz de confusión** (Ref 5): muestra cuántas instancias fueron correcta o incorrectamente clasificadas.
- **Reporte de clasificación** (Ref 6): incluye precisión, exhaustividad (recall), F1-score para cada clase.
- **Accuracy promedio (10-CV)**: proporción de instancias correctamente clasificadas.
- **AUC promedio (10-CV)**: mide la capacidad del modelo para distinguir entre clases (aunque este modelo no tiene capacidad real de discriminación, el valor sirve como base comparativa).

```
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Crear el clasificador Dummy
dummy = DummyClassifier(strategy='most_frequent')

# Definir la validación cruzada (StratifiedKFold)
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Realizar validación cruzada y obtener predicciones
y_pred = cross_val_predict(dummy, X, y, cv=cv)

# Mostrar la matriz de confusión
print("\nMatriz de Confusión :")
print(confusion_matrix(y, y_pred))

# Mostrar el reporte de clasificación
print("Reporte de clasificación:")
print(classification_report(y, y_pred, zero_division=0))

# Calcular y mostrar AUC y Accuracy con 10-CV
acc_cv = cross_val_score(dummy, X, y, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(dummy, X, y, cv=cv, scoring='roc_auc')

print(f"\nDummyClassifier - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"DummyClassifier - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")
```

```

Matriz de Confusión :
[[218334    0]
 [ 35346    0]]
Reporte de clasificación:
      precision    recall  f1-score   support

     0       0.86       1.00       0.93       218334
     1       0.00       0.00       0.00        35346

 accuracy          0.86       253680
 macro avg       0.43       0.50       0.46       253680
 weighted avg    0.74       0.86       0.80       253680

```

```

DummyClassifier - Accuracy promedio (10-CV): 0.860666982024598 ± 1.931165044763883e-05
DummyClassifier - AUC promedio (10-CV): 0.5 ± 0.0

```

Este análisis muestra que el modelo acierta al predecir la clase mayoritaria (sin diabetes), pero no tiene valor predictivo real, ya que no identifica a la clase minoritaria (diabetes). Esto se refleja en un AUC bajo y métricas deficientes. Par analizar los resultados obtenidos se ha empleado la ayuda de un modelo de lenguaje desarrollado por OpenAI (2025) (Ref 7).

La matriz de confusión indica que el modelo predijo correctamente que 218.334 personas no tienen diabetes, pero clasificó erróneamente a 35.346 personas con diabetes como no diabéticas.

En el reporte de clasificación, el **recall** para la clase 1 (diabetes) es 0, lo que significa que el modelo no identifica a ningún caso de diabetes. El recall mide qué porcentaje de los casos reales positivos (personas con diabetes) han sido correctamente detectados. Un valor de 0 implica que el modelo no detecta ninguno.

El **F1-score** combina precisión y recall en una sola métrica. Es útil especialmente cuando hay un desbalance de clases, como en este caso. El F1-score para la clase 0 (sin diabetes) es 0,93, lo que indica un buen equilibrio entre precisión y recall. Sin embargo, para la clase 1 es 0, lo que confirma que el modelo no aporta nada útil para predecir casos de diabetes.

El **accuracy** (exactitud) es de 0,8607, pero este valor es engañoso porque el modelo solo acierta gracias a la clase mayoritaria. Finalmente, el **AUC** de 0,5 indica que el modelo no tiene capacidad real para distinguir entre personas con y sin diabetes; su rendimiento es equivalente al azar.

2.2.2. LogisticRegression

Se implementó un modelo de Regresión Logística (Ref 8 y Ref 9) como primer clasificador real tras el modelo base. Se ha utilizado el mismo procedimiento que con el clasificador anterior, utilizando validación cruzada estratificada (10-CV) para obtener predicciones y evaluar el rendimiento del modelo con métricas como accuracy y AUC. Además, se generaron la matriz de confusión y el reporte de clasificación para analizar el comportamiento del modelo en términos de precisión, sensibilidad y F1-score. Esto permite comparar directamente su rendimiento con el clasificador base y con futuros modelos más complejos.

```

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo de regresión logística
logreg = LogisticRegression(max_iter=1000)

# Obtener predicciones con validación cruzada
y_pred = cross_val_predict(logreg, X, y, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - Logistic Regression:")
print(confusion_matrix(y, y_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - Logistic Regression:")
print(classification_report(y, y_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(logreg, X, y, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(logreg, X, y, cv=cv, scoring='roc_auc')

print(f"\nLogistic Regression - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Logistic Regression - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")

```

Matriz de Confusión - Logistic Regression:

```

[[214701  3633]
 [ 31262  4084]]

```

Reporte de Clasificación - Logistic Regression:

	precision	recall	f1-score	support
0	0.87	0.98	0.92	218334
1	0.53	0.12	0.19	35346
accuracy			0.86	253680
macro avg	0.70	0.55	0.56	253680
weighted avg	0.83	0.86	0.82	253680

```

Logistic Regression - Accuracy promedio (10-CV): 0.862444812362031 ± 0.0010801656530490804
Logistic Regression - AUC promedio (10-CV): 0.799812601504774 ± 0.004345422126647894

```

El modelo de Logistic Regression mejora notablemente respecto al DummyClassifier. Logra identificar correctamente a muchas personas sin diabetes y, aunque aún falla bastante con los casos positivos, empieza a detectar algunos. El **recall** para la clase con diabetes sigue siendo bajo (0,12), pero ya no es nulo como en el Dummy. El **f1-score** también mejora ligeramente (0,19), reflejando un desempeño limitado pero existente en esa clase.

Los resultados muestran una mejora significativa en el **AUC**, que se incrementa hasta 0,7998, mientras que el **accuracy** se mantiene en 0,8624. Esto indica que, aunque el modelo clasifica correctamente una proporción similar de ejemplos, mejora considerablemente en su capacidad para distinguir entre clases, especialmente en contextos con desequilibrio de clases.

2.2.3. RandomForestClassifier

Después de probar con regresión logística, también quise probar un modelo un poco más avanzado: Random Forest. Este modelo es muy utilizado porque combina muchos árboles de decisión y suele dar buenos resultados, incluso cuando los datos son algo complejos. Igual que con los otros modelos, apliqué validación cruzada con 10 divisiones para asegurarme de que los resultados fueran fiables y mostré los mismos resultados. (Ref 10)

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo Random Forest
rf = RandomForestClassifier(n_estimators=10, random_state=42)

# Obtener predicciones con validación cruzada
y_pred = cross_val_predict(rf, X, y, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - Random Forest:")
print(confusion_matrix(y, y_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - Random Forest:")
print(classification_report(y, y_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(rf, X, y, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(rf, X, y, cv=cv, scoring='roc_auc')

print(f"\nRandom Forest - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Random Forest - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")
```

Matriz de Confusión - Random Forest:

```
[[206252 12082]
 [ 28116  7230]]
```

Reporte de Clasificación - Random Forest:

	precision	recall	f1-score	support
0	0.88	0.94	0.91	218334
1	0.37	0.20	0.26	35346
accuracy			0.84	253680
macro avg	0.63	0.57	0.59	253680
weighted avg	0.81	0.84	0.82	253680

```
Random Forest - Accuracy promedio (10-CV): 0.8415405234941659 ± 0.0015501334023801429
Random Forest - AUC promedio (10-CV): 0.7078964157297766 ± 0.002760746312619812
```

El modelo de Random Forest también mejora respecto al DummyClassifier, aunque su comportamiento es distinto al de la regresión logística. La matriz de confusión muestra que identificó correctamente a 7.230 personas con diabetes, un número superior al de Logistic Regression, aunque aún clasificó erróneamente a 28.116.

Esto se refleja en un **recall** de 0,20 para la clase positiva, y un **f1-score** de 0,26, mejores que los de Logistic, pero aún bajos.

Los resultados del Random Forest muestran que se obtuvo un **accuracy** de 0,8415, lo que indica que el modelo acierta en la mayoría de las predicciones. Sin embargo, el **AUC** fue de 0,7079, un valor más bajo en comparación con la regresión logística. Esto significa que, aunque Random Forest clasifica bien en general, distingue con menor precisión entre personas con y sin diabetes. Es decir, le cuesta más identificar correctamente los casos positivos frente a los negativos.

En resumen, Random Forest muestra un desempeño algo más equilibrado en la detección de casos positivos, a costa de una ligera pérdida de exactitud general.

2.2.4. Comparación de los clasificadores básicos

	Accuracy (10-CV)	AUC (10-CV)	Precision		Recall		F1-Score	
			Clase 0	Clase 1	Clase 0	Clase 1	Clase 0	Clase 1
Dummy Classifier	0,8607	0,5	0,86	0,00	1,00	0,00	0,93	0,00
Logistic Regression (sin ajuste)	0,8624	0,7998	0,87	0,53	0,98	0,12	0,92	0,19
Random Forest (sin ajuste)	0,8415	0,7079	0,88	0,37	0,94	0,20	0,91	0,26

El **Dummy Classifier** tiene un accuracy del 86,07% pero predice siempre la clase mayoritaria (sin diabetes), lo que lo hace ineficaz para identificar a las personas con diabetes, con precision y recall nulos para la clase 1. Tiene un AUC de 0,5 lo que indica que es tan bueno como una predicción aleatoria.

La **regresión logística** tiene un accuracy de 86,24% y un AUC de 0,7998, pero sufre de un bajo recall para la clase 1 (0,12), lo que indica que no identifica bien a las personas con diabetes, a pesar de tener un buen rendimiento en la clase 0.

El **Random Forest** presenta un accuracy de 84,15% y un AUC de 0,7079. Aunque mejora ligeramente el recall para la clase 1 (0,20) en comparación con la regresión logística, aún tiene dificultades significativas para identificar a las personas con diabetes

Todos los modelos tienen un accuracy similar (aproximadamente 86%), pero el recall y F1-score para la clase 1 (diabetes) siguen siendo bajos, lo que indica un problema de desbalance de clases. El Dummy Classifier claramente no es útil, mientras que Logistic Regression y Random Forest muestran un desempeño más adecuado, aunque ambos necesitan mejoras en la identificación de la clase minoritaria (diabetes). Una manera de mejorar los modelos podría ser ajustando los hiperparametros para manejar mejor el desbalance de clases.

2.3. Ajuste de los parámetros

Una vez evaluados los modelos básicos, el siguiente paso consiste en ajustar los parámetros (hiperparámetros) de los clasificadores para intentar mejorar su rendimiento, especialmente en términos de AUC, que es una métrica clave esta práctica. Para ello, se va a utilizar **RandomizedSearchCV**, que permite encontrar la mejor combinación de parámetros mediante validación cruzada. Esta técnica selecciona aleatoriamente un número limitado de combinaciones dentro de un espacio definido de parámetros. (Ref 11)

Esto permite ahorrar tiempo computacional, especialmente cuando el número de parámetros es grande, manteniendo una buena capacidad para encontrar configuraciones óptimas. El ajuste se realizó utilizando validación cruzada (10-CV) para asegurar que los resultados fuesen estables y no dependieran de una sola partición de los datos.

El objetivo es obtener modelos que no solo acierten más, sino que también sean más eficaces al identificar correctamente a las personas con diabetes.

Antes de ajustar los hiperparámetros de los modelos, se realizó una división del conjunto de datos en dos partes: un **70% para entrenamiento** y un **30% para prueba** o, en inglés, test. Esta separación permite entrenar el modelo con una parte de los datos y luego evaluarlo con otra parte independiente, asegurando así que los resultados no estén influidos por sobreajuste.

```
from sklearn.model_selection import train_test_split

# X: características (features)
# y: etiquetas (labels)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)
```

2.3.1. Ajuste para LogisticRegression

Para llevar a cabo el ajuste de hiperparámetros mediante RandomizedSearchCV, primero hubo que definir el espacio de hiperparámetros a explorar (Ref 12). Los parámetros que se ajustaron en el modelo de LogisticRegression fueron:

- **C:** Este parámetro controla la regularización. Un valor pequeño indica una mayor regularización y un valor alto implica una menor regularización. Para este ajuste, utilicé una distribución logarítmica entre 0,001 y 100.
- **penalty:** El tipo de penalización aplicada. Usé dos opciones: 'l1' y 'l2', que son las formas comunes de regularización.
- **solver:** El algoritmo utilizado para optimizar la regresión logística. Las opciones fueron 'liblinear' y 'saga', siendo 'saga' más adecuado para grandes conjuntos de datos y penalizaciones l1.
- **max_iter:** El número máximo de iteraciones para que el algoritmo converja. Este valor se ajustó dentro de un rango entre 100 y 2000.
- **tol:** Es la tolerancia para la convergencia, es decir, la precisión deseada en el proceso de optimización.

Con estos parámetros definidos, utilicé el conjunto de entrenamiento `X_train` e `y_train` para realizar el ajuste de hiperparámetros. `RandomizedSearchCV` evaluó varias combinaciones aleatorias de estos hiperparámetros utilizando validación cruzada en 5 particiones (`cv=5`) para asegurarse de que el modelo generalizara bien. (Ref 13)

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RandomizedSearchCV, cross_val_score
from scipy.stats import loguniform, randint
import numpy as np

# Definir el modelo base
logreg = LogisticRegression(random_state=42)

# Espacio de hiperparámetros a explorar
param_distributions = {
    'C': loguniform(0.001, 100),      # Regularización (usando una distribución logarítmica)
    'penalty': ['l1', 'l2'],          # Tipo de penalización
    'solver': ['liblinear', 'saga'],   # Algoritmos de optimización
    'max_iter': randint(100, 2000),   # Número máximo de iteraciones
    'tol': loguniform(1e-5, 1e-1),    # Tolerancia para la convergencia
}

# Configurar RandomizedSearchCV
random_search_logreg = RandomizedSearchCV(
    estimator=logreg,
    param_distributions=param_distributions,
    n_iter=15,                        # 15 combinaciones aleatorias
    scoring='roc_auc',               # AUC como métrica de optimización
    n_jobs=-1,                       # Usa todos los núcleos de CPU
    cv=5,                            # Validación cruzada de 5 particiones
    random_state=42,
    verbose=2
)

# Ejecutar la búsqueda de los mejores hiperparámetros
y_train = y_train.values.ravel() # Convertir a vector unidimensional si es necesario
random_search_logreg.fit(X_train, y_train)

# Mostrar los mejores hiperparámetros encontrados
print("Mejores hiperparámetros encontrados para Regresión Logística:")
print(random_search_logreg.best_params_)

# Entrenar el modelo final con los mejores hiperparámetros
best_logreg = random_search_logreg.best_estimator_

# Evaluar el modelo final con validación cruzada (10 folds)
auc_scores_logreg = cross_val_score(best_logreg, X, y, cv=10, scoring='roc_auc')

# Mostrar el resultado final de AUC
print(f"AUC promedio del modelo de Regresión Logística final (10-CV): {np.mean(auc_scores_logreg)} ± {np.std(auc_scores_logreg)}")
```

Al final del proceso, `RandomizedSearchCV` encontró los mejores valores de los hiperparámetros, lo que resultó en un modelo optimizado de `LogisticRegression`. Los mejores hiperparámetros encontrados fueron:

- `C`: 0,00498
- `max_iter`: 1599
- `penalty`: 'l2'
- `solver`: 'saga'
- `tol`: 8,53e-5

Después de ajustar los hiperparámetros con `RandomizedSearchCV`, el modelo de regresión logística muestra un rendimiento muy similar al modelo sin ajustar.

La matriz de confusión y el reporte de clasificación muestran un desempeño similar al del modelo sin ajustar. Aunque el modelo ajustado tiene un **accuracy** del 86 %, sigue teniendo problemas significativos para identificar correctamente a las personas con diabetes, lo cual se refleja en un **recall** bajo para la clase 1 (0,11) y un **f1-score** igualmente bajo (0,18) para la misma clase.

El **AUC** promedio también es similar a la obtenida sin ajuste de hiperparámetros. La AUC de 0,8000 presenta solo una ligera mejora respecto al modelo base.

Esto puede ser indicativo de que, aunque los hiperparámetros ajustados pueden tener cierto impacto en el modelo, el desbalance de clases sigue siendo un desafío importante.

Matriz de Confusión - Logistic Regression:

```
[[214887  3447]
 [ 31485  3861]]
```

Reporte de Clasificación - Logistic Regression:

	precision	recall	f1-score	support
0	0.87	0.98	0.92	218334
1	0.53	0.11	0.18	35346
accuracy			0.86	253680
macro avg	0.70	0.55	0.55	253680
weighted avg	0.82	0.86	0.82	253680

Logistic Regression - Accuracy promedio (10-CV): 0.8622989593188268 ± 0.0009978412820708291
Logistic Regression - AUC promedio (10-CV): 0.7999836726648197 ± 0.004284921912840723

2.3.2. Ajuste para RandomForestClassifier

Para ajustar los hiperparámetros del modelo Random Forest utilicé también RandomizedSearchCV. Lo primero fue definir el espacio de hiperparámetros a explorar (Ref 14). Los parámetros que seleccioné para ajustar fueron:

- **n_estimators:** Es el número de árboles que componen el bosque. Cuantos más árboles tenga, mejor suele ser el rendimiento, aunque también aumenta el tiempo de ejecución. Para este ajuste, seleccioné un rango entre 100 y 300.
- **max_depth:** Indica la profundidad máxima que puede tener cada árbol. En este caso, exploré valores entre 5 y 13.
- **min_samples_split:** Es el número mínimo de muestras necesarias para dividir un nodo. Valores más altos hacen que el árbol sea más conservador. Utilicé un rango entre 2 y 20.
- **min_samples_leaf:** Representa el número mínimo de muestras que debe haber en una hoja del árbol. También usé un rango entre 1 y 20.
- **max_features:** Controla cuántas características se consideran al buscar la mejor división en un nodo. Probé con 'sqrt', 'log2' y None.
- **bootstrap:** Determina si los árboles se construyen con muestreo con reemplazo (bootstrap) o sin él. Probé tanto True como False.

Con todos estos parámetros definidos, utilicé el conjunto de entrenamiento X_train e y_train para ejecutar el ajuste de hiperparámetros. RandomizedSearchCV probó 15 combinaciones aleatorias de los hiperparámetros y evaluó su rendimiento usando validación cruzada con 5 particiones (cv=5).

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV, cross_val_score
from scipy.stats import randint
import numpy as np

# Definir el modelo base
rf = RandomForestClassifier(random_state=42)

# Espacio de hiperparámetros a explorar
param_distributions = {
    'n_estimators': randint(100, 300),      # número de árboles
    'max_depth': randint(5, 13),            # profundidad máxima
    'min_samples_split': randint(2, 20),    # mínimo de muestras para dividir un nodo
    'min_samples_leaf': randint(1, 20),     # mínimo de muestras en una hoja
    'max_features': ['sqrt', 'log2', None], # máximo número de características consideradas
    'bootstrap': [True, False]             # usar bootstrap o no
}

# Configurar RandomizedSearchCV
random_search_rf = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_distributions,
    n_iter=15,          # 15 combinaciones aleatorias
    scoring='roc_auc',
    n_jobs=-1,
    cv=5,               # validación cruzada de 5 particiones
    random_state=42,
    verbose=2
)

# Ejecutar la búsqueda de los mejores hiperparámetros
y_train = y_train.values.ravel() # Convertir a vector unidimensional si es necesario
random_search_rf.fit(X_train, y_train)

# Mostrar los mejores hiperparámetros
print("Mejores hiperparámetros encontrados para Random Forest:")
print(random_search_rf.best_params_)

# Entrenar el modelo final con los mejores hiperparámetros
best_rf = random_search_rf.best_estimator_

# Evaluar el modelo final
auc_scores_rf = cross_val_score(best_rf, X, y, cv=10, scoring='roc_auc')
print(f"AUC promedio del Random Forest final (10-CV): {np.mean(auc_scores_rf)} ± {np.std(auc_scores_rf)}")

```

Al final del proceso, RandomizedSearchCV encontró los mejores valores de los hiperparámetros, lo que resultó en un modelo optimizado de RandomForestClassifier. Los mejores hiperparámetros encontrados fueron:

- bootstrap = True
- max_depth = 11
- max_features = 'log2'
- min_samples_leaf = 4
- min_samples_split = 15
- n_estimators = 108

Después de ajustar los hiperparámetros con RandomizedSearchCV, el modelo de Random Forest muestra un rendimiento mejorado en comparación con el modelo sin ajustar. Para la clase 0, el modelo acierta bien, con un **recall** de 0,99. La clase 1 tiene el recall muy bajo (0,09), indicando que solo el 9% de las personas con diabetes son correctamente identificadas. Esto se ve también reflejado en el **f1-score** que en la clase 0 tiene un valor elevado (buen equilibrio entre la precisión y el recall) y en la clase 1 el valor es bajo.

El **accuracy** se sigue manteniendo en un 86% aproximadamente mientras que el **AUC** aumenta de 0,7079 cuando el clasificador no tenía los parámetros ajustados a un 0,8059 ahora que se han ajustado. Esto refleja una clara mejora en el rendimiento del modelo, lo que significa que el ajuste de hiperparámetros ha tenido un efecto positivo, mejorando la capacidad del modelo para diferenciar entre personas con y sin diabetes.

Matriz de Confusión - Random Forest:

```
[[216061  2273]
 [ 32052 3294]]
```

Reporte de Clasificación - Random Forest:

	precision	recall	f1-score	support
0	0.87	0.99	0.93	218334
1	0.59	0.09	0.16	35346
accuracy			0.86	253680
macro avg	0.73	0.54	0.54	253680
weighted avg	0.83	0.86	0.82	253680

Random Forest - Accuracy promedio (10-CV): 0.8646917376222012 ± 0.0008669206881095162
Random Forest - AUC promedio (10-CV): 0.8059473339594673 ± 0.005182605194371904

2.3.3. Comparación de los clasificadores básicos con hiperparámetros

	Accuracy (10-CV)	AUC (10-CV)	Precision		Recall		F1-Score	
			Clase 0	Clase 1	Clase 0	Clase 1	Clase 0	Clase 1
Logistic Regression (sin ajuste)	0,8624	0,7998	0,87	0,53	0,98	0,12	0,92	0,19
Logistic Regression (con ajuste)	0,8623	0,8000	0,87	0,53	0,98	0,11	0,92	0,18
Random Forest (sin ajuste)	0,8415	0,7079	0,88	0,37	0,94	0,20	0,91	0,26
Random Forest (con ajuste)	0,8647	0,8059	0,87	0,59	0,99	0,09	0,93	0,16

Tras el ajuste de hiperparámetros, el modelo de **regresión logística** mantiene un rendimiento casi idéntico al obtenido sin ajustar.

En cambio, el modelo de **Random Forest** muestra una ligera mejora general tras el ajuste. El AUC mejora bastante ya que sube de 0,7079 a 0,8059. También aumenta la precisión para la clase 1 (de 0,37 a 0,59), aunque el recall disminuye ligeramente (de 0,20 a 0,09). Esto indica que el modelo se vuelve más conservador en sus predicciones positivas, pero acierta más cuando predice un caso positivo.

En comparación, ambos modelos ajustados logran un rendimiento similar en términos de accuracy, pero Random Forest supera a la regresión logística en AUC y precisión para la clase minoritaria. Además, el ajuste de hiperparámetros tiene un impacto más evidente en el modelo Random Forest, lo que sugiere que es más sensible a este tipo de optimización.

2.4. Balanceo de clases

Dado que el conjunto de datos presenta un importante desbalance entre las clases (con una proporción mucho mayor de personas no diabéticas que diabéticas), se aplicaron técnicas de balanceo únicamente sobre el conjunto de entrenamiento, tal como se recomienda en la práctica. El objetivo de este paso es evitar introducir sesgos en la evaluación de los modelos. En este caso, se utilizaron dos enfoques diferentes: oversampling y undersampling.

En concreto, se utilizará SMOTE para generar nuevos ejemplos sintéticos de la clase minoritaria (personas con diabetes). Para el método de undersampling se empleará uno aleatorio llamado RandomUnderSampler para reducir la cantidad de ejemplos de la clase mayoritaria.

2.4.1. SMOTE (oversampling)

En primer lugar, se optó por utilizar la técnica SMOTE (Synthetic Minority Over-sampling Technique) debido al desbalance significativo entre las clases. SMOTE genera ejemplos sintéticos de la clase minoritaria (personas con diabetes) a partir de los vecinos más cercanos de los datos existentes. De este modo, se logra un conjunto de entrenamiento más equilibrado sin recurrir a la duplicación de ejemplos.

Esta técnica se aplicó únicamente al conjunto de entrenamiento, ya que es importante evitar modificar el conjunto de prueba para garantizar una evaluación justa del modelo. Tras aplicar SMOTE, se entrenaron dos modelos: Logistic Regression y Random Forest. Para cada uno de ellos, se evaluaron los resultados con y sin ajuste de hiperparámetros, con el objetivo de analizar si el uso de SMOTE mejora el rendimiento de los modelos y si el ajuste de los hiperparámetros tiene un impacto adicional en los resultados. Todos los modelos se evaluaron utilizando el conjunto de prueba original, que no fue alterado.

2.4.1.1. SMOTE LogisticRegression

Para comenzar con el balanceo de LogisticRegression se debe aplicar SMOTE sobre el conjunto de entrenamiento (X_{train} e y_{train}). El resultado es un conjunto de entrenamiento balanceado.

Posteriormente se entrena el modelo de regresión logística utilizando este nuevo conjunto de entrenamiento. En esta parte del código es donde se deben indicar los hiperparámetros del clasificador si es que se desean tener en cuenta. En esta parte de la práctica se va a estudiar el sobremuestreo del clasificador tanto sin parámetros como con ellos.

Después de entrenar el modelo, se realiza la predicción sobre el conjunto de prueba (X_{test}), que no ha sido modificado. A continuación, se genera la matriz de confusión y el reporte de clasificación. El parámetro `zero_division = 0` asegura que no se generen errores si alguna métrica tiene una división por cero.

Finalmente, se realiza una validación cruzada de 10 priegues para evaluar el modelo utilizando el accuracy y el área bajo la curva ROC (AUC).

Códigos creados siguiendo ejemplos presentes en la referencia 15.

Código sin hiperparámetros:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import cross_val_score
from imblearn.over_sampling import SMOTE

# 1. SMOTE sobre el conjunto de entrenamiento
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
y_train_smote = y_train_smote.values.ravel()

# 2. Entrenar el modelo
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train_smote, y_train_smote)

# 3. Predecir sobre el conjunto de test original
y_pred = logreg.predict(X_test)

# 4. Matriz de confusión
conf_mat = confusion_matrix(y_test, y_pred)
print("Matriz de Confusión:\n", conf_mat)

# 5. Reporte de clasificación
report = classification_report(y_test, y_pred, zero_division=0)
print("\nReporte de Clasificación:\n", report)

# 6. 10-CV para Accuracy
acc_cv = cross_val_score(logreg, X_train_smote, y_train_smote, cv=10, scoring='accuracy')

# 7. 10-CV para AUC
auc_cv = cross_val_score(logreg, X_train_smote, y_train_smote, cv=10, scoring='roc_auc')

print(f"\nLogistic Regression - Accuracy promedio (10-CV) con SMOTE: {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Logistic Regression - AUC promedio (10-CV) con SMOTE: {auc_cv.mean()} ± {auc_cv.std()}")
```

Código con hiperparámetros:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import cross_val_score
from imblearn.over_sampling import SMOTE
import numpy as np

# 1. SMOTE sobre el conjunto de entrenamiento
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
y_train_smote = y_train_smote.values.ravel()

# 2. Entrenar el modelo
logreg = LogisticRegression(
    C=np.float64(0.004982752357076452),
    max_iter=1599,
    penalty='l2',
    solver='saga',
    tol=np.float64(8.532678095658718e-05)
)

logreg.fit(X_train_smote, y_train_smote)

# 3. Predecir sobre el conjunto de test original
y_pred = logreg.predict(X_test)

# 4. Matriz de confusión
conf_mat = confusion_matrix(y_test, y_pred)
print("Matriz de Confusión:\n", conf_mat)

# 5. Reporte de clasificación
report = classification_report(y_test, y_pred, zero_division=0)
print("\nReporte de Clasificación:\n", report)

# 6. 10-CV para Accuracy
acc_cv = cross_val_score(logreg, X_train_smote, y_train_smote, cv=10, scoring='accuracy')

# 7. 10-CV para AUC
auc_cv = cross_val_score(logreg, X_train_smote, y_train_smote, cv=10, scoring='roc_auc')

print(f"\nLogistic Regression - Accuracy promedio (10-CV) con SMOTE: {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Logistic Regression - AUC promedio (10-CV) con SMOTE: {auc_cv.mean()} ± {auc_cv.std()}")
```

Resultados sin hiperparámetros:

Matriz de Confusión:

```
[[45434 20171]
 [ 2574 7925]]
```

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.95	0.69	0.80	65605
1	0.28	0.75	0.41	10499
accuracy			0.70	76104
macro avg	0.61	0.72	0.61	76104
weighted avg	0.85	0.70	0.75	76104

Logistic Regression - Accuracy promedio (10-CV) con SMOTE: 0.7184948384751374 ± 0.003262395882086003
Logistic Regression - AUC promedio (10-CV) con SMOTE: 0.7885868553735684 ± 0.002868448874418567

Antes de aplicar SMOTE, el modelo de regresión logística alcanzaba un **recall** para la clase 1 de apenas 0,12, lo que indica que la mayoría de los casos positivos no eran detectados. Tras aplicar SMOTE, ese valor sube a 0,75, una mejora notable. Aunque la **precisión** baja de 0,53 a 0,28, esta pérdida es esperable al forzar al modelo a ser más sensible a la clase minoritaria. En términos de **f1-score** para la clase 1, también se ve una mejora, pasando de 0,19 a 0,41. El **AUC** se mantiene en un buen nivel aunque el **accuracy** haya caído significativamente.

Esto confirma que SMOTE ayuda al modelo a centrarse más en detectar correctamente los casos positivos, aunque el precio a pagar sea un aumento de falsos positivos.

Resultados con hiperparámetros:

Matriz de Confusión:

```
[[45469 20136]
 [ 2571 7928]]
```

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.95	0.69	0.80	65605
1	0.28	0.76	0.41	10499
accuracy			0.70	76104
macro avg	0.61	0.72	0.61	76104
weighted avg	0.85	0.70	0.75	76104

Logistic Regression - Accuracy promedio (10-CV) con SMOTE: 0.7185570386735731 ± 0.003317788987480004
Logistic Regression - AUC promedio (10-CV) con SMOTE: 0.7886684405191349 ± 0.0032113815627620697

Los resultados muestran que el ajuste de hiperparámetros no produce una diferencia significativa respecto al modelo sin ajustar, una vez que se ha aplicado SMOTE. El **recall** para la clase 1 se mantiene prácticamente igual (0,76 frente a 0,75), y el **f1-score** también se conserva en 0,41. La **accuracy** global y la **AUC** apenas varían respecto a la versión sin ajuste.

Esto sugiere que, en este caso concreto, el uso de SMOTE tiene un mayor impacto en el rendimiento del modelo que el propio ajuste de hiperparámetros.

En la siguiente tabla se recogen los resultados del clasificador de regresión logística obtenidos hasta el momento:

LOGISTIC REGRESSION	Accuracy (10-CV)	AUC (10-CV)	Precision		Recall		F1-Score	
			Clase 0	Clase 1	Clase 0	Clase 1	Clase 0	Clase 1
Logistic Regression (sin ajuste)	0,8624	0,7998	0,87	0,53	0,98	0,12	0,92	0,19
Logistic Regression (con ajuste)	0,8623	0,8000	0,87	0,53	0,98	0,11	0,92	0,18
Logistic Regression (SMOTE, sin ajuste)	0,7185	0,7886	0,95	0,28	0,69	0,75	0,80	0,41
Logistic Regression (SMOTE, con ajuste)	0,7186	0,7887	0,95	0,28	0,69	0,76	0,80	0,41

2.4.1.2. SMOTE RandomForestClassifier

Para el clasificador RandomForestClassifier, se sigue exactamente el mismo procedimiento que con la regresión logística: se aplica SMOTE únicamente sobre el conjunto de entrenamiento para equilibrar las clases, se entrena el modelo con o sin ajuste de hiperparámetros, y se evalúa sobre el conjunto de prueba original. A continuación, se presentan los códigos utilizados para ambas configuraciones. Códigos creados siguiendo ejemplos presentes en la referencia 15.

Código sin hiperparámetros:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import cross_val_score
from imblearn.over_sampling import SMOTE

# 1. SMOTE sobre el conjunto de entrenamiento
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
y_train_smote = y_train_smote.values.ravel()

# 2. Entrenar el modelo
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train_smote, y_train_smote)

# 3. Predecir sobre el conjunto de test original
y_pred = rf_model.predict(X_test)

# 4. Matriz de confusión
conf_mat = confusion_matrix(y_test, y_pred)
print("Matriz de Confusión:\n", conf_mat)

# 5. Reporte de clasificación
report = classification_report(y_test, y_pred, zero_division=0)
print("\nReporte de Clasificación:\n", report)

# 6. 10-CV para Accuracy
acc_cv = cross_val_score(rf_model, X_train_smote, y_train_smote, cv=10, scoring='accuracy')

# 7. 10-CV para AUC
auc_cv = cross_val_score(rf_model, X_train_smote, y_train_smote, cv=10, scoring='roc_auc')

print(f"\nRandom Forest - Accuracy promedio (10-CV) con SMOTE: {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Random Forest - AUC promedio (10-CV) con SMOTE: {auc_cv.mean()} ± {auc_cv.std()}")
```

Código con hiperparámetros:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import cross_val_score
from imblearn.over_sampling import SMOTE

# 1. SMOTE sobre el conjunto de entrenamiento
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
y_train_smote = y_train_smote.values.ravel()

# 2. Entrenar el modelo
rf_model = RandomForestClassifier(
    bootstrap=True,
    max_depth=11,
    max_features='log2',
    min_samples_leaf=4,
    min_samples_split=15,
    n_estimators=100,
    random_state=42
)
rf_model.fit(X_train_smote, y_train_smote)

# 3. Predecir sobre el conjunto de test original
y_pred = rf_model.predict(X_test)

# 4. Matriz de confusión
conf_mat = confusion_matrix(y_test, y_pred)
print("Matriz de Confusión:\n", conf_mat)

# 5. Reporte de clasificación
report = classification_report(y_test, y_pred, zero_division=0)
print("\nReporte de Clasificación:\n", report)

# 6. 10-CV para Accuracy
acc_cv = cross_val_score(rf_model, X_train_smote, y_train_smote, cv=10, scoring='accuracy')

# 7. 10-CV para AUC
auc_cv = cross_val_score(rf_model, X_train_smote, y_train_smote, cv=10, scoring='roc_auc')

print(f"\nRandom Forest - Accuracy promedio (10-CV) con SMOTE: {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Random Forest - AUC promedio (10-CV) con SMOTE: {auc_cv.mean()} ± {auc_cv.std()}")
```

Resultados sin hiperparámetros:

Matriz de Confusión:

```
[[50067 15538]
 [ 5169  5330]]
```

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.91	0.76	0.83	65605
1	0.26	0.51	0.34	10499
accuracy			0.73	76104
macro avg	0.58	0.64	0.58	76104
weighted avg	0.82	0.73	0.76	76104

```
Random Forest - Accuracy promedio (10-CV) con SMOTE: 0.8084385065920399 ± 0.010310499214734791
Random Forest - AUC promedio (10-CV) con SMOTE: 0.8876166722679347 ± 0.009460544475296722
```

Antes de aplicar SMOTE, el modelo Random Forest alcanzaba un **recall** para la clase 1 de solo 0,20. Tras aplicar la técnica de sobremuestreo, ese valor aumenta hasta 0,51, lo que implica que el modelo es capaz de detectar una mayor proporción de casos positivos.

También mejora ligeramente el **f1-score** de la clase 1 (de 0,26 a 0,34), aunque la **precisión** baja (de 0,37 a 0,26), algo esperable al aumentar la sensibilidad.

El **accuracy** total también sube (de 0,8415 a 0,73), y especialmente mejora el **AUC** (de 0,7079 a 0,8876), lo que indica una mejor discriminación general del modelo entre ambas clases.

Resultados con hiperparámetros:

```
Matriz de Confusión:
[[45494 20111]
 [ 2499  8000]]

Reporte de Clasificación:
      precision    recall  f1-score   support

     0       0.95      0.69      0.80     65605
     1       0.28      0.76      0.41     10499

 accuracy          0.70      0.70      0.70     76104
  macro avg       0.62      0.73      0.61     76104
 weighted avg     0.86      0.70      0.75     76104
```

Random Forest - Accuracy promedio (10-CV) con SMOTE: 0.7345494241933279 ± 0.002333228423424302
Random Forest - AUC promedio (10-CV) con SMOTE: 0.8096286626585629 ± 0.002145016036740924

Al ajustar hiperparámetros tras aplicar SMOTE, se observa un cambio en el comportamiento del modelo. El **recall** para la clase 1 sigue mejorando ligeramente (hasta 0,76), y el **f1-score** sube a 0,41, igualando el resultado obtenido en Logistic Regression tras SMOTE. Sin embargo, la **precisión** se mantiene baja (0,28) y el **accuracy** global cae en comparación con el modelo sin ajuste (de 0,73 a 0,70). Además, el **AUC** baja de 0,8876 a 0,8096. Esto sugiere que, en este caso, el ajuste de hiperparámetros no mejora el rendimiento general del modelo.

En la siguiente tabla se recogen los resultados del clasificador Random Forest obtenidos hasta el momento:

RANDOM FOREST	Accuracy (10-CV)	AUC (10-CV)	Precision		Recall		F1-Score	
			Clase 0	Clase 1	Clase 0	Clase 1	Clase 0	Clase 1
Random Forest (sin ajuste)	0,8415	0,7079	0,88	0,37	0,94	0,20	0,91	0,26
Random Forest (con ajuste)	0,8647	0,8059	0,87	0,59	0,99	0,09	0,93	0,16
Random Forest (SMOTE, sin ajuste)	0,8084	0,8876	0,91	0,26	0,76	0,51	0,83	0,34
Random Forest (SMOTE, con ajuste)	0,7345	0,8096	0,95	0,28	0,69	0,76	0,80	0,41

2.4.2. RandomUnderSampler (undersampling)

Otra estrategia explorada para abordar el desbalance de clases ha sido el *undersampling*, concretamente utilizando la técnica de RandomUnderSampler (Ref 16). A diferencia del oversampling, que genera nuevas instancias de la clase minoritaria, el undersampling elimina aleatoriamente ejemplos de la clase mayoritaria para equilibrar el conjunto.

Este enfoque se aplica, como en el caso anterior, solo sobre el conjunto de entrenamiento, con el objetivo de no alterar la distribución original del conjunto de prueba y así mantener una evaluación realista del modelo.

En esta sección se han entrenado los clasificadores de LogisticRegression y RandomForestClassifier utilizando conjuntos de datos balanceados mediante undersampling, tanto sin ajuste como con ajuste de hiperparámetros.

2.4.2.1. RandomUnderSampler LogisticRegression

Primero, se ha aplicado el recorte aleatorio de la clase mayoritaria en el conjunto de entrenamiento, lo que da lugar a un dataset equilibrado entre positivos y negativos. A partir de este conjunto reducido, se entrena el modelo.

Tal como se hizo anteriormente con SMOTE, se han probado dos configuraciones: una sin ajuste de hiperparámetros y otra con ajuste. Para medir los resultados obtenidos de cada estrategia se han calculado las métricas de clasificación, como la matriz de confusión, el reporte de clasificación, y también se ha llevado a cabo una validación cruzada de 10 pliegues para obtener valores medios de accuracy y AUC.

Se presentan los diferentes códigos a ejecutar. Se sigue el mismo esquema que con SMOTE pero cambiando éste por RandomUnderSampler. Código sin hiperparámetros:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import cross_val_score
from imblearn.under_sampling import RandomUnderSampler

# 1. RandomUnderSampler sobre el conjunto de entrenamiento
undersampler = RandomUnderSampler(random_state=42)
X_train_undersampled, y_train_undersampled = undersampler.fit_resample(X_train, y_train)
y_train_undersampled = y_train_undersampled.values.ravel()

# 2. Entrenar el modelo
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train_undersampled, y_train_undersampled)

# 3. Predecir sobre el conjunto de test original
y_pred = logreg.predict(X_test)

# 4. Matriz de confusión
conf_mat = confusion_matrix(y_test, y_pred)
print("Matriz de Confusión:\n", conf_mat)

# 5. Reporte de clasificación
report = classification_report(y_test, y_pred, zero_division=0)
print("\nReporte de Clasificación:\n", report)

# 6. 10-CV para Accuracy
acc_cv = cross_val_score(logreg, X_train_undersampled, y_train_undersampled, cv=10, scoring='accuracy')

# 7. 10-CV para AUC
auc_cv = cross_val_score(logreg, X_train_undersampled, y_train_undersampled, cv=10, scoring='roc_auc')

print(f"\nLogistic Regression - Accuracy promedio (10-CV) con RandomUnderSampler: {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Logistic Regression - AUC promedio (10-CV) con RandomUnderSampler: {auc_cv.mean()} ± {auc_cv.std()}")
```

Código con hiperparámetros:

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import cross_val_score
from imblearn.under_sampling import RandomUnderSampler

# 1. RandomUnderSampler sobre el conjunto de entrenamiento
undersampler = RandomUnderSampler(random_state=42)
X_train_undersampled, y_train_undersampled = undersampler.fit_resample(X_train, y_train)
y_train_undersampled = y_train_undersampled.values.ravel()

# 2. Entrenar el modelo
logreg = LogisticRegression(
    C=np.float64(0.004982752357076452),
    max_iter=1599,
    penalty='l2',
    solver='saga',
    tol=np.float64(8.532678095658718e-05)
)
logreg.fit(X_train_undersampled, y_train_undersampled)

# 3. Predecir sobre el conjunto de test original
y_pred = logreg.predict(X_test)

# 4. Matriz de confusión
conf_mat = confusion_matrix(y_test, y_pred)
print("Matriz de Confusión:\n", conf_mat)

# 5. Reporte de clasificación
report = classification_report(y_test, y_pred, zero_division=0)
print("\nReporte de Clasificación:\n", report)

# 6. 10-CV para Accuracy
acc_cv = cross_val_score(logreg, X_train_undersampled, y_train_undersampled, cv=10, scoring='accuracy')

# 7. 10-CV para AUC
auc_cv = cross_val_score(logreg, X_train_undersampled, y_train_undersampled, cv=10, scoring='roc_auc')

print(f"\nLogistic Regression - Accuracy promedio (10-CV) con RandomUnderSampler: {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Logistic Regression - AUC promedio (10-CV) con RandomUnderSampler: {auc_cv.mean()} ± {auc_cv.std()}")

```

Resultados sin hiperparámetros:

Matriz de Confusión:

```
[[46504 19101]
 [ 2613  7886]]
```

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.95	0.71	0.81	65605
1	0.29	0.75	0.42	10499
accuracy			0.71	76104
macro avg	0.62	0.73	0.62	76104
weighted avg	0.86	0.71	0.76	76104

```

Logistic Regression - Accuracy promedio (10-CV) con RandomUnderSampler: 0.730651767315505 ± 0.005434771444145392
Logistic Regression - AUC promedio (10-CV) con RandomUnderSampler: 0.8043857910827752 ± 0.00589615624335402

```

Después de aplicar RandomUnderSampler, se observa una mejora clara en el **recall** de la clase 1 respecto al modelo original sin balanceo, pasando de 0,12 a 0,75. Esto significa que el modelo consigue identificar muchos más casos positivos. La **precisión**, en cambio, cae de 0,53 a 0,29, como es habitual al dar más peso a la clase minoritaria. El **f1-score** para la clase 1 mejora hasta 0,42, confirmando que el modelo logra un equilibrio más razonable entre sensibilidad y precisión.

Tanto el **accuracy** como el **AUC** obtenidos en la validación cruzada también reflejan una mejora general con respecto al modelo original no balanceado (el AUC pasa de 0,7998 a 0,8044), aunque ligeramente inferiores a los alcanzados con SMOTE.

Resultados con hiperparámetros:

Matriz de Confusión:
[[46430 19175]
[2587 7912]]

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.95	0.71	0.81	65605
1	0.29	0.75	0.42	10499
accuracy			0.71	76104
macro avg	0.62	0.73	0.62	76104
weighted avg	0.86	0.71	0.76	76104

Logistic Regression - Accuracy promedio (10-CV) con RandomUnderSampler: 0.7312353574050462 ± 0.005844544709938604
Logistic Regression - AUC promedio (10-CV) con RandomUnderSampler: 0.80403687558204 ± 0.005677704560971763

La versión ajustada del modelo muestra un rendimiento muy similar al anterior, con valores prácticamente iguales en todas las métricas. El **recall** de la clase 1 se mantiene en 0,75 y el **f1-score** en 0,42. El **accuracy** y el **AUC** medios apenas varían (0,7312 y 0,8040 respectivamente), lo que sugiere que, al igual que ocurrió con SMOTE, en este caso el ajuste de hiperparámetros no aporta una mejora significativa adicional tras haber balanceado las clases con undersampling.

La tabla presentada a continuación contiene un resumen de todos los resultados de las métricas realizadas con Logistic Regression desde el inicio del estudio hasta ahora:

	Accuracy (10-CV)	AUC (10-CV)	Precision		Recall		F1-Score	
			Clase 0	Clase 1	Clase 0	Clase 1	Clase 0	Clase 1
LOGISTIC REGRESSION								
Sin ajuste de hiperparámetros	0,8624	0,7998	0,87	0,53	0,98	0,12	0,92	0,19
Con ajuste de hiperparámetros	0,8623	0,8000	0,87	0,53	0,98	0,11	0,92	0,18
Sin ajuste y con SMOTE	0,7185	0,7886	0,95	0,28	0,69	0,75	0,80	0,41
Con ajuste y con SMOTE	0,7186	0,7887	0,95	0,28	0,69	0,76	0,80	0,41
Sin ajuste y con RandomUnderSampler	0.7307	0.8044	0.95	0.29	0.71	0.75	0.81	0.42
Con ajuste y con RandomUnderSampler	0.7312	0.8040	0.95	0.29	0.71	0.75	0.81	0.42

2.4.2.2. RandomUnderSampler RandomForestClassifier

Para el clasificador RandomForestClassifier, se sigue exactamente el mismo procedimiento que con la regresión logística: se aplica RandomUnderSampler únicamente sobre el conjunto de entrenamiento para equilibrar las clases, se entrena el modelo con o sin ajuste de hiperparámetros, y se evalúa sobre el conjunto de prueba original.

A continuación, se muestran los códigos y resultados obtenidos.

Código sin hiperparámetros:

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import cross_val_score
from imblearn.under_sampling import RandomUnderSampler

# 1. RandomUnderSampler sobre el conjunto de entrenamiento
undersampler = RandomUnderSampler(random_state=42)
X_train_undersampled, y_train_undersampled = undersampler.fit_resample(X_train, y_train)
y_train_undersampled = y_train_undersampled.values.ravel()

# 2. Entrenar el modelo
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train_undersampled, y_train_undersampled)

# 3. Predecir sobre el conjunto de test original
y_pred = rf_model.predict(X_test)

# 4. Matriz de confusión
conf_mat = confusion_matrix(y_test, y_pred)
print("Matriz de Confusión:\n", conf_mat)

# 5. Reporte de clasificación
report = classification_report(y_test, y_pred, zero_division=0)
print("\nReporte de Clasificación:\n", report)

# 6. 10-CV para Accuracy
acc_cv = cross_val_score(rf_model, X_train_undersampled, y_train_undersampled, cv=10, scoring='accuracy')

# 7. 10-CV para AUC
auc_cv = cross_val_score(rf_model, X_train_undersampled, y_train_undersampled, cv=10, scoring='roc_auc')

print(f"\nRandom Forest - Accuracy promedio (10-CV) con RandomUnderSampler: {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Random Forest - AUC promedio (10-CV) con RandomUnderSampler: {auc_cv.mean()} ± {auc_cv.std()}")

```

Código con hiperparámetros:

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import cross_val_score
from imblearn.under_sampling import RandomUnderSampler

# 1. RandomUnderSampler sobre el conjunto de entrenamiento
undersampler = RandomUnderSampler(random_state=42)
X_train_undersampled, y_train_undersampled = undersampler.fit_resample(X_train, y_train)
y_train_undersampled = y_train_undersampled.values.ravel()

# 2. Entrenar el modelo
rf_model = RandomForestClassifier(
    bootstrap=True,
    max_depth=11,
    max_features='log2',
    min_samples_leaf=4,
    min_samples_split=15,
    n_estimators=108,
    random_state=42
)
rf_model.fit(X_train_undersampled, y_train_undersampled)

# 3. Predecir sobre el conjunto de test original
y_pred = rf_model.predict(X_test)

# 4. Matriz de confusión
conf_mat = confusion_matrix(y_test, y_pred)
print("Matriz de Confusión:\n", conf_mat)

# 5. Reporte de clasificación
report = classification_report(y_test, y_pred, zero_division=0)
print("\nReporte de Clasificación:\n", report)

# 6. 10-CV para Accuracy
acc_cv = cross_val_score(rf_model, X_train_undersampled, y_train_undersampled, cv=10, scoring='accuracy')

# 7. 10-CV para AUC
auc_cv = cross_val_score(rf_model, X_train_undersampled, y_train_undersampled, cv=10, scoring='roc_auc')

print(f"\nRandom Forest - Accuracy promedio (10-CV) con RandomUnderSampler: {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Random Forest - AUC promedio (10-CV) con RandomUnderSampler: {auc_cv.mean()} ± {auc_cv.std()}")

```

Resultados sin hiperparámetros:

Matriz de Confusión:

```
[[44583 21022]
 [ 3174  7325]]
```

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.93	0.68	0.79	65605
1	0.26	0.70	0.38	10499
accuracy			0.68	76104
macro avg	0.60	0.69	0.58	76104
weighted avg	0.84	0.68	0.73	76104

Random Forest - Accuracy promedio (10-CV) con RandomUnderSampler: 0.6918139385720643 ± 0.004644892834859763
Random Forest - AUC promedio (10-CV) con RandomUnderSampler: 0.7545883841689379 ± 0.005127956486387234

En el caso de RandomForestClassifier sin ajuste, la aplicación de RandomUnderSampler mejora considerablemente el **recall** para la clase minoritaria (clase 1), pasando de valores muy bajos previos al balanceo hasta alcanzar 0,70. No obstante, esta ganancia en sensibilidad viene acompañada de una pérdida en **precisión**, que cae hasta 0,26. El **f1-score** para la clase 1 se eleva a 0,38, lo que indica un mejor equilibrio entre precisión y recall que antes del balanceo.

Además, el **accuracy** total baja hasta 0,6918, reflejando que el modelo sacrifica rendimiento general para centrarse más en la clase minoritaria. El **AUC** alcanza 0,7546, lo cual es aceptable y muestra que, en promedio, el modelo discrimina bien entre clases.

Resultados con hiperparámetros:

Matriz de Confusión:

```
[[45322 20283]
 [ 2358  8141]]
```

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.95	0.69	0.80	65605
1	0.29	0.78	0.42	10499
accuracy			0.70	76104
macro avg	0.62	0.73	0.61	76104
weighted avg	0.86	0.70	0.75	76104

Random Forest - Accuracy promedio (10-CV) con RandomUnderSampler: 0.7331873794588825 ± 0.005011482401920303
Random Forest - AUC promedio (10-CV) con RandomUnderSampler: 0.8084880578715573 ± 0.004791030199809346

El ajuste de hiperparámetros mejora ligeramente los resultados respecto al modelo sin ajustar. El **recall** para la clase 1 sube a 0,78 y el **f1-score** se incrementa hasta 0,42. Además, el **accuracy** y el **AUC** también mejoran (0,73 y 0,81 respectivamente), mostrando que el modelo ajustado consigue un mejor rendimiento global sin sacrificar la capacidad para detectar la clase minoritaria.

Aunque el modelo sin ajustar ya mejora respecto a no balancear, el ajuste fino permite aprovechar mejor el muestreo, aumentando la sensibilidad sin un deterioro excesivo de la precisión.

La tabla presentada a continuación contiene un resumen de todos los resultados de las métricas realizadas con Random Forest desde el inicio del estudio hasta ahora:

RANDOM FOREST	Accuracy (10-CV)	AUC (10-CV)	Precision		Recall		F1-Score	
			Clase 0	Clase 1	Clase 0	Clase 1	Clase 0	Clase 1
Sin ajuste de hiperparámetros	0,8415	0,7079	0,88	0,37	0,94	0,20	0,91	0,26
Con ajuste de hiperparámetros	0,8647	0,8059	0,87	0,59	0,99	0,09	0,93	0,16
Sin ajuste y con SMOTE	0,8084	0,8876	0,91	0,26	0,76	0,51	0,83	0,34
Con ajuste y con SMOTE	0,7345	0,8096	0,95	0,28	0,69	0,76	0,80	0,41
Sin ajuste y con RandomUnderSampler	0,6918	0,7546	0,93	0,26	0,68	0,70	0,79	0,38
Con ajuste y con RandomUnderSampler	0,7332	0,8085	0,95	0,29	0,69	0,78	0,80	0,42

2.4.3. Comparación de los clasificadores básicos tras el balanceo

A lo largo de la práctica, hemos probado dos técnicas de balanceo de clases: SMOTE (oversampling) y RandomUnderSampler (undersampling), tanto en Logistic Regression como en Random Forest, con y sin ajuste de hiperparámetros. A continuación, se presenta una comparación de los resultados:

- **SMOTE** (oversampling) es más efectivo para mejorar el recall de la clase minoritaria, especialmente en Random Forest, aunque puede aumentar los falsos positivos, lo que reduce la precisión.
- **RandomUnderSampler** (undersampling) también mejora el recall de la clase 1, pero de forma menos significativa en Random Forest. Además, en Logistic Regression, los resultados con y sin ajuste de hiperparámetros son casi idénticos.
- Random Forest se beneficia más del balanceo de clases, especialmente con SMOTE, lo que sugiere que tiene mayor capacidad para adaptarse a los datos balanceados.

En resumen, SMOTE ha mostrado los mejores resultados, especialmente con Random Forest, mientras que el ajuste de hiperparámetros ha tenido un impacto menor.

IMPORTANTE: Después de analizar las diferentes estrategias, **la combinación de RandomForestClassifier sin ajuste y SMOTE ofrece el mejor rendimiento**, logrando el mayor AUC de 0,8876.

2.5. Test de contraste de hipótesis

Para comparar los resultados de la validación cruzada entre los modelos básicos se ha llevado a cabo un test de contraste de hipótesis utilizando el **Wilcoxon Signed-Rank Test** (Ref 17). Este test sirve para comprobar si las diferencias en el rendimiento de dos modelos son estadísticamente significativas o no, en este caso usando el AUC como métrica de referencia.

En este caso, se compararon los dos clasificadores estudiados hasta el momento (Logistic Regression y Random Forest), ambos **sin ajuste de hiperparámetros** ni aplicación de técnicas de balanceo (SMOTE o undersampling). El objetivo es analizar si alguno de los dos clasificadores ofrece un rendimiento significativamente superior al otro en condiciones base, sin alteraciones externas en los datos.

Se utilizó validación cruzada de 10 particiones (10-CV) para obtener las puntuaciones de AUC en cada pliegue para ambos modelos. A continuación, se aplicó el test de Wilcoxon sobre los pares de resultados obtenidos. El nivel de significancia fijado fue de 0,05. Según el valor p resultante, se concluye si existen o no diferencias estadísticamente significativas entre los clasificadores comparados.

```
from scipy.stats import wilcoxon
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

# Instanciar los clasificadores
logreg = LogisticRegression(max_iter=1000)
rf_model = RandomForestClassifier(random_state=42)

# Realizar la validación cruzada para obtener los resultados de AUC de ambos modelos
logreg_auc = cross_val_score(logreg, X, y.ravel(), cv=10, scoring='roc_auc')
rf_auc = cross_val_score(rf_model, X, y.ravel(), cv=10, scoring='roc_auc')

# Realizar el test de Wilcoxon para comparar los resultados de AUC
stat, p_value = wilcoxon(logreg_auc, rf_auc)

# Mostrar los resultados del test
print(f"Wilcoxon Signed-Rank Test - Estadístico: {stat}")
print(f"Wilcoxon Signed-Rank Test - p-valor: {p_value}")

# Interpretar el p-valor
alpha = 0.05 # Nivel de significancia

if p_value < alpha:
    print("Rechazamos la hipótesis nula: hay diferencias significativas entre los modelos.")
else:
    print("No rechazamos la hipótesis nula: no hay diferencias significativas entre los modelos.")

print(f"Logistic Regression - AUC promedio: {np.mean(logreg_auc)}")
print(f"Random Forest - AUC promedio: {np.mean(rf_auc)}")
```

Estos fueron los resultados obtenidos:

```
Wilcoxon Signed-Rank Test - Estadístico: 0.0
Wilcoxon Signed-Rank Test - p-valor: 0.001953125
Rechazamos la hipótesis nula: hay diferencias significativas entre los modelos.
Logistic Regression - AUC promedio: 0.7997845995807972
Random Forest - AUC promedio: 0.72982876358812
```

Para poder comprender los resultados se consultó la referencia 18. El test de Wilcoxon ha obtenido un estadístico de 0.0 y un p-valor de 0.00195, por lo que se rechaza la hipótesis nula al nivel de significancia del 5%. Esto indica que existen diferencias significativas entre los dos modelos, y que no se deben al azar.

En concreto, el modelo de Logistic Regression ha obtenido un AUC promedio de 0.7998, mientras que Random Forest alcanzó un valor más bajo, de 0.7298. Por tanto, en este escenario sin ajuste ni balanceo, Logistic Regression ofrece un mejor rendimiento en términos de AUC.

Después de comparar ambos modelos sin ajuste, he repetido el mismo procedimiento estadístico **con ajuste de hiperparámetros** para evaluar éste produce diferencias significativas entre Logistic Regression y Random Forest. Al igual que antes, he utilizado validación cruzada de 10 pliegues y la métrica AUC como criterio de evaluación.

Se ha aplicado el test de Wilcoxon sobre los AUC obtenidos por cada modelo para comprobar si las diferencias observadas tras el ajuste son estadísticamente significativas. A continuación, se muestran los resultados obtenidos.

```
from scipy.stats import wilcoxon
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
import numpy as np

# Instanciar los clasificadores
logreg = LogisticRegression(
    C=np.float64(0.004982752357076452),
    max_iter=1599,
    penalty='l2',
    solver='saga',
    tol=np.float64(8.532678095658718e-05)
)
rf_model = RandomForestClassifier(
    bootstrap=True,
    max_depth=11,
    max_features='log2',
    min_samples_leaf=4,
    min_samples_split=15,
    n_estimators=100,
    random_state=42
)

# Realizar la validación cruzada para obtener los resultados de AUC de ambos modelos
y = y.values.ravel()
logreg_auc = cross_val_score(logreg, X, y, cv=10, scoring='roc_auc')
rf_auc = cross_val_score(rf_model, X, y, cv=10, scoring='roc_auc')

# Realizar el test de Wilcoxon para comparar los resultados de AUC
stat, p_value = wilcoxon(logreg_auc, rf_auc)

# Mostrar los resultados del test
print(f"Wilcoxon Signed-Rank Test - Estadístico: {stat}")
print(f"Wilcoxon Signed-Rank Test - p-valor: {p_value}")

# Interpretar el p-valor
alpha = 0.05 # Nivel de significancia

if p_value < alpha:
    print("Rechazamos la hipótesis nula: hay diferencias significativas entre los modelos.")
else:
    print("No rechazamos la hipótesis nula: no hay diferencias significativas entre los modelos.")

print(f"Logistic Regression - AUC promedio: {np.mean(logreg_auc)}")
print(f"Random Forest - AUC promedio: {np.mean(rf_auc)}")
```

Estos fueron los resultados obtenidos:

```
Wilcoxon Signed-Rank Test - Estadístico: 0.0  
Wilcoxon Signed-Rank Test - p-valor: 0.001953125  
Rechazamos la hipótesis nula: hay diferencias significativas entre los modelos.  
Logistic Regression - AUC promedio: 0.7999571493630013  
Random Forest - AUC promedio: 0.8059473339594673
```

Después de realizar el test de Wilcoxon, se ha calculado un p-valor de 0.00195, por lo que se rechaza la hipótesis nula y se concluye que existen diferencias significativas entre ambos modelos. En cuanto a los resultados de AUC promedio, Random Forest obtiene un valor ligeramente superior (0.8059) frente a Logistic Regression (0.7999), lo que sugiere que, con ajuste, el rendimiento de RandomForestClassifier es mejor significativamente respecto al otro clasificador.

En conclusión, con respecto al test de Wilcoxon, se puede observar un cambio interesante entre las versiones sin y con ajuste de hiperparámetros. Sin ajuste, Logistic Regression supera claramente a Random Forest, obteniendo un AUC promedio mayor y con diferencias estadísticamente significativas. Sin embargo, al aplicar el ajuste de hiperparámetros, la situación se invierte: Random Forest mejora notablemente su rendimiento, superando por poco a Logistic Regression. Esto demuestra que Random Forest se beneficia más del ajuste y que su rendimiento depende en mayor medida de una buena configuración de parámetros.

3. NIVEL MEDIO

3.1. Algoritmos de selección automática de atributos relevantes

La selección automática de atributos tiene como objetivo reducir la dimensionalidad, es decir, reducir el número de características (o variables) que se utilizan para entrenar un modelo sin perder demasiada información relevante. La reducción de la dimensionalidad ayuda a mejorar la eficiencia computacional, reducir el sobreajuste (overfitting) y, en algunos casos, mejorar el rendimiento del modelo.

Para reducir la dimensionalidad y seleccionar las variables más relevantes, se pueden utilizar diversos métodos de selección de características entre los que destacan el SelectKBest y el Recursive Feature Elimination (RFE).

3.1.1. SelectKBest

SelectKBest, que es una técnica sencilla y eficiente para la selección de características. (Ref 19) Este método permite elegir las mejores características de un conjunto de datos en función de una métrica estadística que mide la relación entre las características y la variable objetivo.

Su objetivo es identificar las características que realmente aportan información útil, eliminando las irrelevantes. Esto se consigue un procedimiento de selección basado en pruebas estadísticas, como el test de chi-cuadrado o la correlación.

Al inicio de la práctica, se realizó una revisión de las variables explicativas para identificar cuáles podrían ser prescindibles. Ahora, se quiere verificar si las características que fueron eliminadas eran importantes para el modelo. Para ello, se vuelve a cargar la base de datos y a aplicar el algoritmo SelectKBest sobre la tabla de características (X) para evaluar la relevancia de cada una de ellas y comprobar si las eliminadas afectaban significativamente al rendimiento del modelo. De esta forma, se podrá validar si la selección inicial de variables fue adecuada. Parte del código fue desarrollado con la asistencia de ChatGPT, una herramienta de inteligencia artificial proporcionada por OpenAI (Ref 7).

```
[ ] # Cargar dataset original completo
cdc_diabetes_health_indicators = fetch_ucirepo(id=891)
X_select = cdc_diabetes_health_indicators.data.features
y_select = cdc_diabetes_health_indicators.data.targets

import pandas as pd
from sklearn.feature_selection import SelectKBest, f_classif
from ucimlrepo import fetch_ucirepo

y_select = y_select.values.ravel()

# Aplicar SelectKBest con ANOVA F-test (f_classif)
selector = SelectKBest(score_func=f_classif, k='all') # 'all' para puntuarlas todas
selector.fit(X_select, y_select)

# Crear ranking de variables
scores = selector.scores_
feature_scores = pd.DataFrame({
    'Feature': X_select.columns,
    'Score': scores
}).sort_values(by='Score', ascending=False)

print("Ranking de variables según SelectKBest:")
print(feature_scores)
```

Estos fueron los resultados:

```
Ranking de variables según SelectKBest:
      Feature      Score
13  GenHlth  23924.564885
0   HighBP  18870.365816
16  DiffWalk 12699.341579
3    BMI     12516.718642
1   HighChol 10600.350806
18   Age     8246.866284
6  HeartDiseaseorAttack 8231.555129
15  PhysHlth  7672.267690
20   Income  7004.370724
19  Education 3991.111142
7   PhysActivity 3590.290347
5    Stroke    2872.607437
14  MentHlth  1224.700591
2   CholCheck  1068.396844
4    Smoker    940.878480
10  HvyAlcoholConsump  828.524705
9    Veggies    814.826113
8    Fruits    422.555361
12  NoDocbcCost  250.886466
17   Sex       250.842280
11  AnyHealthcare    67.046943
```

Tras aplicar SelectKBest, se puede concluir que el análisis realizado inicialmente para eliminar variables de la base de datos no fue completamente adecuado. Se observa que la variable '**GenHlth**', que fue descartada al inicio del estudio, es la más relevante con diferencia, obteniendo la puntuación más alta en el ranking. Además, algunas de las variables previamente eliminadas, como '**Education**' e '**Income**', también están entre las más relevantes según el algoritmo. Esto sugiere que la eliminación de estas variables pudo haber afectado negativamente los resultados obtenidos en las etapas anteriores. Por lo tanto, se recomienda revisar y ajustar la selección de variables y, posiblemente, repetir el análisis con una nueva base de datos, teniendo en cuenta los resultados del algoritmo de selección automática y sin eliminar estas variables clave.

3.1.2. Recursive Feature Elimination (RFE)

Por otro lado, Recursive Feature Elimination (RFE) es otro enfoque para seleccionar las mejores características, pero con una metodología diferente. En lugar de elegir un número fijo de características como se hace en el algoritmo anteriormente estudiado, RFE utiliza un modelo de aprendizaje automático para evaluar la importancia de cada característica de forma iterativa. El propósito de RFE es ir eliminando de manera recursiva las características menos relevantes para el modelo, lo que permite no solo reducir la dimensionalidad del conjunto de datos, sino también mejorar la precisión al centrarse solo en las variables clave (Ref 20).

Se van a utilizar distintos clasificadores ya que pueden dar diferentes rankings porque cada uno entiende y modela los datos de forma diferente. Logistic Regression evalúa relaciones lineales, mientras que Random Forest captura relaciones no lineales y considera interacciones entre variables.

3.1.2.1. LogisticRegression

En este caso, se ha utilizado un modelo de **Logistic Regression** como estimador base, ya que es un clasificador lineal sencillo y efectivo que permite evaluar la importancia de cada variable según sus coeficientes.

El código mostrado comienza importando las librerías necesarias y luego se asegura de que `y_select` sea un array unidimensional. Después se define el clasificador a utilizar y se crea un objeto RFE, indicando que se desean seleccionar las **10 variables más importantes** (`n_features_to_select=10`) y, posteriormente, se ajusta el modelo con los datos (`rfe.fit()`). Finalmente se extraen e imprimen las variables seleccionadas y un ranking completo .

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
import pandas as pd
from ucimlrepo import fetch_ucirepo

# Asegúrate de que y_select sea un array 1D
y_select = y_select.values.ravel()

# Modelo base para RFE
logreg = LogisticRegression(max_iter=1000)
```

```

# Aplicar RFE para seleccionar, por ejemplo, las 10 mejores variables
rfe = RFE(estimator=logreg, n_features_to_select=10)
rfe.fit(X_select, y_select)

# Ver qué variables fueron seleccionadas
selected_features = X_select.columns[rfe.support_]
ranking = pd.DataFrame({
    'Feature': X_select.columns,
    'Selected': rfe.support_,
    'Ranking': rfe.ranking_
}).sort_values('Ranking')

print("Variables seleccionadas por RFE:")
print(selected_features)
print("\nRanking completo:")
print(ranking)

```

Los resultados impresos son los siguientes:

```

Variables seleccionadas por RFE:
Index(['HighBP', 'HighChol', 'CholCheck', 'HeartDiseaseorAttack', 'Fruits',
      'HvyAlcoholConsump', 'GenHlth', 'DiffWalk', 'Sex', 'Age'],
      dtype='object')

```

Ranking completo:

	Feature	Selected	Ranking
0	HighBP	True	1
1	HighChol	True	1
2	CholCheck	True	1
6	HeartDiseaseorAttack	True	1
10	HvyAlcoholConsump	True	1
13	GenHlth	True	1
8	Fruits	True	1
16	DiffWalk	True	1
17	Sex	True	1
18	Age	True	1
5	Stroke	False	2
3	BMI	False	3
11	AnyHealthcare	False	4
20	Income	False	5
7	PhysActivity	False	6
9	Veggies	False	7
19	Education	False	8
4	Smoker	False	9
12	NoDocbcCost	False	10
15	PhysHlth	False	11
14	MentHlth	False	12

Las 10 variables seleccionadas por RFE incluyen factores tanto médicos como demográficos y de estilo de vida. Estas variables fueron clasificadas con un ranking igual a 1, lo que indica que son las más útiles para la regresión logística en este contexto.

Por otro lado, otras variables no fueron seleccionadas y presentan valores de ranking superiores, lo que sugiere que aportan menos información relevante al modelo.

Es importante destacar que la variable 'GenHlth', también señalada como clave en otros métodos de selección, vuelve a aparecer como una de las más significativas, reforzando su relevancia.

3.1.2.2. RandomForestClassifier

En este apartado se aplica el algoritmo de Recursive Feature Elimination (RFE) utilizando RandomForestClassifier como estimador base.

A diferencia de los modelos lineales como Logistic Regression, los modelos basados en árboles como Random Forest pueden capturar relaciones no lineales y manejar mejor la interacción entre variables, por lo que pueden ofrecer un enfoque complementario para la selección de atributos.

El procedimiento seguido con Random Forest como estimador ha sido exactamente el mismo que en el caso anterior.

```
from sklearn.feature_selection import RFE
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
from ucimlrepo import fetch_ucirepo

# Asegúrate de que y_select sea un array 1D
y_select = y_select.values.ravel()

# Modelo base para RFE
rf = RandomForestClassifier(random_state=42)

# Aplicar RFE para seleccionar, por ejemplo, las 10 mejores variables
rfe = RFE(estimator=rf, n_features_to_select=10)
rfe.fit(X_select, y_select)

# Ver qué variables fueron seleccionadas
selected_features = X_select.columns[rfe.support_]
ranking = pd.DataFrame({
    'Feature': X_select.columns,
    'Selected': rfe.support_,
    'Ranking': rfe.ranking_
}).sort_values('Ranking')

print("Variables seleccionadas por RFE con Random Forest:")
print(selected_features)
print("\nRanking completo:")
print(ranking)
```

Estos son los resultados:

```
Variables seleccionadas por RFE con Random Forest:
Index(['HighBP', 'BMI', 'Smoker', 'Fruits', 'GenHlth', 'MentHlth', 'PhysHlth',
      'Age', 'Education', 'Income'],
      dtype='object')
```


Ranking completo:

	Feature	Selected	Ranking
0	HighBP	True	1
3	BMI	True	1
4	Smoker	True	1
8	Fruits	True	1
13	GenHlth	True	1
14	MentHlth	True	1
15	PhysHlth	True	1
19	Education	True	1
18	Age	True	1
20	Income	True	1
9	Veggies	False	2
16	DiffWalk	False	3
17	Sex	False	4
1	HighChol	False	5
7	PhysActivity	False	6
6	HeartDiseaseorAttack	False	7
12	NoDocbcCost	False	8
5	Stroke	False	9
11	AnyHealthcare	False	10
10	HvyAlcoholConsump	False	11
2	CholCheck	False	12

A diferencia de la selección obtenida con Logistic Regression, aquí aparecen algunas variables que habían sido eliminadas en fases previas del análisis, como ‘MentHlth’, ‘PhysHlth’, ‘Education’ o ‘Income’, lo que resalta la importancia de considerar diferentes enfoques para la selección de características. Además, variables como ‘Smoker’ o ‘BMI’, que no fueron consideradas relevantes por el modelo lineal, sí lo son desde la perspectiva de Random Forest. Esto refuerza la idea de que el tipo de clasificador utilizado puede influir significativamente en la interpretación de la relevancia de las variables, y sugiere la conveniencia de combinar distintas técnicas de selección para tomar decisiones más robustas.

3.2. Clasificadores medios

Para abordar la parte correspondiente a los clasificadores del nivel medio, se ha decidido trabajar con una nueva versión del conjunto de variables explicativas. Esta segunda tabla ha sido construida a partir de los resultados obtenidos mediante las técnicas automáticas de selección de características, empleadas en el apartado anterior, con el objetivo de mejorar la calidad del modelo y reducir la dimensionalidad del problema.

En esta nueva tabla de características se han eliminado las siguientes variables basándose en los siguientes criterios:

- **Smoker:** Tiene una puntuación baja en SelectKBest (940.87) y un ranking poco favorable en RFE con Logistic Regression (ranking 9), por lo que se considera poco relevante.
- **Stroke:** Presenta un bajo score en SelectKBest y un ranking alto tanto en RFE con Logistic Regression como con Random Forest, indicando escasa importancia en ambos casos.

- **AnyHealthcare:** Con un score bajo en SelectKBest y ranking 4 en RFE (Logistic Regression), no parece aportar información significativa.
- **NoDocbcCost:** Muy baja puntuación en SelectKBest y un ranking alto en RFE, lo que sugiere un impacto limitado en el modelo.
- **PhysActivity:** Score bajo en SelectKBest y mal posicionada en RFE con Random Forest, lo que refuerza su baja relevancia.
- **Veggies:** Puntuación modesta en SelectKBest y ranking desfavorable en RFE (Random Forest), también es considerada prescindible.
- **CholCheck:** Aunque tiene una puntuación algo más alta en SelectKBest, en RFE con Logistic Regression obtiene un ranking de 12 (de 12), lo que indica que su influencia es mínima en ese modelo.

De este modo, se procede a la eliminación de las variables no seleccionadas de la tabla X2.

```
[ ] # Cargar dataset original completo
cdc_diabetes_health_indicators = fetch_ucirepo(id=891)
X2 = cdc_diabetes_health_indicators.data.features
y2 = cdc_diabetes_health_indicators.data.targets

[ ] X2 = X2.drop(columns=[
    "Stroke", "AnyHealthcare", "NoDocbcCost", "PhysActivity",
    "Veggies", "CholCheck", "Smoker",
])

[ ] from sklearn.model_selection import train_test_split

# X: características (features)
# y: etiquetas (labels)
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.30, random_state=42)
```

Esta tabla depurada de variables permitirá trabajar con un conjunto más reducido, pero potencialmente más informativo de características, lo que puede contribuir a mejorar la eficiencia del modelo y a obtener resultados más robustos en los clasificadores de nivel medio.

3.2.1. LogisticRegression

En esta sección se procede a aplicar nuevamente el clasificador Logistic Regression, pero esta vez utilizando la nueva tabla de variables explicativas (X2).

El propósito de este nuevo estudio es evaluar si el uso de un subconjunto optimizado de variables puede mejorar el rendimiento del modelo, reducir el riesgo de sobreajuste y simplificar su interpretación. Se seguirán los mismos pasos que en el análisis inicial: validación cruzada y evaluación de métricas de rendimiento, especialmente el AUC. Esta comparación permitirá determinar si el proceso de selección de características ha tenido un impacto positivo en la calidad del clasificador.

```

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo de regresión logística
logreg = LogisticRegression(max_iter=1000)

# Obtener predicciones con validación cruzada
y2 = y2.values.ravel()
y2_pred = cross_val_predict(logreg, X2, y2, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - Logistic Regression:")
print(confusion_matrix(y2, y2_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - Logistic Regression:")
print(classification_report(y2, y2_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(logreg, X2, y2, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(logreg, X2, y2, cv=cv, scoring='roc_auc')

print(f"\nLogistic Regression - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Logistic Regression - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")

```

Matriz de Confusión - Logistic Regression:

```
[[213605  4729]
 [ 29911  5435]]
```

Reporte de Clasificación - Logistic Regression:

	precision	recall	f1-score	support
0	0.88	0.98	0.92	218334
1	0.53	0.15	0.24	35346
accuracy			0.86	253680
macro avg	0.71	0.57	0.58	253680
weighted avg	0.83	0.86	0.83	253680

```

Logistic Regression - Accuracy promedio (10-CV): 0.8634500157678966 ± 0.0013300293187342408
Logistic Regression - AUC promedio (10-CV): 0.8203890175175875 ± 0.004193627649713261

```

El **accuracy** de la regresión logística muestra un rendimiento consistente entre ambas tablas. El **AUC** con la tabla X es de 0,7998, mientras que con la tabla X2 aumenta a 0,8204, lo que sugiere una ligera mejora en la capacidad de discriminación con la nueva selección de variables.

En ambos casos, el modelo tiene una buena **precision** para la clase 0, pero su **recall** para la clase 1 sigue siendo bajo (antes 0,12 y ahora 0,15), lo que indica que no está identificando correctamente muchos de los casos positivos (clase 1).

3.2.2. RandomForestClassifier

A continuación, se aplicará el procedimiento anterior pero esta vez con Random Forest y la tabla X2, para comparar el rendimiento de este clasificador en las mismas condiciones.

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo Random Forest
rf = RandomForestClassifier(random_state=42)

# Obtener predicciones con validación cruzada
y2 = y2.values.ravel()
y2_pred = cross_val_predict(rf, X2, y2, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - Random Forest:")
print(confusion_matrix(y2, y2_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - Random Forest:")
print(classification_report(y2, y2_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(rf, X2, y2, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(rf, X2, y2, cv=cv, scoring='roc_auc')

print(f"\nRandom Forest - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Random Forest - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")

```

Matriz de Confusión - Random Forest:

```
[[208872  9462]
 [ 28669  6677]]
```

Reporte de Clasificación - Random Forest:

	precision	recall	f1-score	support
0	0.88	0.96	0.92	218334
1	0.41	0.19	0.26	35346
accuracy			0.85	253680
macro avg	0.65	0.57	0.59	253680
weighted avg	0.81	0.85	0.82	253680

```
Random Forest - Accuracy promedio (10-CV): 0.8496885840428886 ± 0.001256302891341126
Random Forest - AUC promedio (10-CV): 0.7472355300768324 ± 0.004420191128392016
```

Random Forest mantiene la accuracy bastante estable. Con la tabla X, la **accuracy** promedio es de 0,84, mientras que con la tabla X2 mejora a 0,85, lo que sugiere que la selección de variables de la tabla X2 tiene un ligero impacto positivo en el rendimiento general del modelo. El **AUC** con Random Forest aumenta de 0,71 en la tabla X a 0,75 con la tabla X2, lo que indica una mejora en la capacidad de discriminación, aunque sigue siendo más bajo que el de Logistic Regression.

Similar a Logistic Regression, Random Forest muestra una **precision** alta para la clase 0 (antes 0,88 y ahora 0,96), pero el **recall** para la clase 1 sigue siendo bajo (antes 0,19 y ahora 0,20), lo que refleja que el modelo no logra identificar bien la clase minoritaria. Aunque la accuracy es alta, la capacidad para identificar correctamente los casos positivos sigue siendo limitada.

3.2.3. DecisionTreeClassifier

Para este estudio, se implementará el clasificador Decision Tree y se aplicará tanto a la tabla de características X como a la tabla X2. En primer lugar, se utilizará la tabla X, en la cual las variables fueron eliminadas según un criterio basado en la investigación de la enfermedad. Este paso permitirá observar cómo el modelo de Decision Tree se comporta con un conjunto de características previamente seleccionadas sin la ayuda de algoritmos automáticos.

Este modelo no lineal construye una estructura jerárquica de decisiones basada en particiones sucesivas de los datos, lo que le permite capturar relaciones complejas entre las variables.

```
[ ] from sklearn.tree import DecisionTreeClassifier
    from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
    from sklearn.metrics import classification_report, confusion_matrix

    # Definir la validación cruzada estratificada
    cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

    # Crear el modelo Decision Tree
    decision_tree = DecisionTreeClassifier()

    # Obtener predicciones con validación cruzada
    y = y.values.ravel()
    y_pred = cross_val_predict(decision_tree, X, y, cv=cv)

    # Matriz de confusión
    print("\nMatriz de Confusión - Decision Tree:")
    print(confusion_matrix(y, y_pred))

    # Reporte de clasificación
    print("\nReporte de Clasificación - Decision Tree:")
    print(classification_report(y, y_pred, zero_division=0))

    # Evaluación con métricas por validación cruzada
    acc_cv = cross_val_score(decision_tree, X, y, cv=cv, scoring='accuracy')
    auc_cv = cross_val_score(decision_tree, X, y, cv=cv, scoring='roc_auc')

    print(f"\nDecision Tree - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
    print(f"Decision Tree - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")
```

Matriz de Confusión - Decision Tree:

```
[[205420 12914]
 [ 28368  6978]]
```

Reporte de Clasificación - Decision Tree:

	precision	recall	f1-score	support
0	0.88	0.94	0.91	218334
1	0.35	0.20	0.25	35346
accuracy			0.84	253680
macro avg	0.61	0.57	0.58	253680
weighted avg	0.81	0.84	0.82	253680

```
Decision Tree - Accuracy promedio (10-CV): 0.837082150741091 ± 0.0012371080557515303
Decision Tree - AUC promedio (10-CV): 0.618052843451014 ± 0.003546831088190832
```

El modelo presenta un buen desempeño al clasificar la clase mayoritaria (0), aunque tiene dificultades importantes para detectar la clase minoritaria (1), como indica su baja **recall** (0,20). Se obtuvo una **accuracy** promedio del 0,8371 y un **AUC** promedio del 0,6181.

A continuación, se aplicará el mismo clasificador sobre la tabla X2, en la que las variables fueron eliminadas basándose en los resultados de algoritmos de selección automática, como SelectKBest o RFE. Al utilizar estos métodos automáticos para seleccionar las variables más relevantes, se espera que el modelo de Decision Tree pueda aprovechar una mejor selección de características, lo que podría influir positivamente en el rendimiento del clasificador.

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo Decision Tree
decision_tree = DecisionTreeClassifier()

# Obtener predicciones con validación cruzada
y2 = y2.ravel()
y2_pred = cross_val_predict(decision_tree, X2, y2, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - Decision Tree:")
print(confusion_matrix(y2, y2_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - Decision Tree:")
print(classification_report(y2, y2_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(decision_tree, X2, y2, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(decision_tree, X2, y2, cv=cv, scoring='roc_auc')

print(f"\nDecision Tree - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Decision Tree - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")

```

Matriz de Confusión - Decision Tree:

```
[[193003  25331]
 [ 24546  10800]]
```

Reporte de Clasificación - Decision Tree:

	precision	recall	f1-score	support
0	0.89	0.88	0.89	218334
1	0.30	0.31	0.30	35346
accuracy			0.80	253680
macro avg	0.59	0.59	0.59	253680
weighted avg	0.81	0.80	0.80	253680

Decision Tree - Accuracy promedio (10-CV): 0.8035162409334594 ± 0.002257908282156554
Decision Tree - AUC promedio (10-CV): 0.5978706136259507 ± 0.004413902030493362

Al aplicar Decision Tree sobre la tabla X2, el rendimiento general fue algo inferior. Se obtuvo una **accuracy** promedio del 0,8035 y un **AUC** de 0,5979. Aunque la **recall** para la clase 1 mejoró ligeramente hasta el 0,31, esto vino acompañado de un incremento en los falsos positivos, lo que penalizó el resultado global del modelo. Además, la capacidad para clasificar correctamente la clase 0 se redujo ligeramente, afectando el equilibrio general.

3.2.4. KNeighborsClassifier

A continuación, se aplica el clasificador KNeighbors, siguiendo el mismo procedimiento que con los modelos anteriores. Este algoritmo se basa en la distancia entre observaciones para asignar una clase, considerando las etiquetas de los vecinos más cercanos. Es especialmente sensible a la escala de los datos y a la presencia de variables irrelevantes, por lo que su rendimiento puede variar significativamente entre las dos tablas de entrada.

Entrenamiento del modelo con la tabla X:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)

# Obtener predicciones con validación cruzada
y = y.ravel()
y_pred = cross_val_predict(knn, X, y, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - KNeighborsClassifier:")
print(confusion_matrix(y, y_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - KNeighborsClassifier:")
print(classification_report(y, y_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(knn, X, y, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(knn, X, y, cv=cv, scoring='roc_auc')

print(f"\nKNeighborsClassifier - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"\nKNeighborsClassifier - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")
```

Matriz de Confusión - KNeighborsClassifier:

```
[[208481  9853]
 [ 28864  6482]]
```

Reporte de Clasificación - KNeighborsClassifier:

	precision	recall	f1-score	support
0	0.88	0.95	0.92	218334
1	0.40	0.18	0.25	35346
accuracy			0.85	253680
macro avg	0.64	0.57	0.58	253680
weighted avg	0.81	0.85	0.82	253680

KNeighborsClassifier - Accuracy promedio (10-CV): 0.847378587196468 ± 0.0015222401574923133
 KNeighborsClassifier - AUC promedio (10-CV): 0.6970684320053854 ± 0.003408450136617543

Con la tabla original, el modelo alcanza una **accuracy** del 0,8474, lo que indica un buen desempeño general. La clase mayoritaria (0) se predice correctamente con una **precisión** del 0,88 y un **recall** del 0,95, mostrando que el modelo es muy eficaz detectando los casos negativos. Sin embargo, como ocurre con muchos clasificadores en problemas desbalanceados, el rendimiento sobre la clase minoritaria (1) es más limitado, con un **recall** del 0,18 y una **f1-score** de 0,25.

El **AUC** promedio (0,6971) refleja una capacidad aceptable del modelo para discriminar entre ambas clases, aunque menor que muchos otros clasificadores.

Ahora entrenamos el modelo con la tabla X2 para comparar el rendimiento antes y después de aplicar la selección automática de variables.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)

# Obtener predicciones con validación cruzada
y2 = y2.ravel()
y2_pred = cross_val_predict(knn, X2, y2, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - KNeighborsClassifier:")
print(confusion_matrix(y2, y2_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - KNeighborsClassifier:")
print(classification_report(y2, y2_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(knn, X2, y2, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(knn, X2, y2, cv=cv, scoring='roc_auc')

print(f"\nKNeighborsClassifier - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"\nKNeighborsClassifier - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")
```

Matriz de Confusión - KNeighborsClassifier:

```
[[207918 10416]
 [ 28388  6958]]
```

Reporte de Clasificación - KNeighborsClassifier:

	precision	recall	f1-score	support
0	0.88	0.95	0.91	218334
1	0.40	0.20	0.26	35346
accuracy			0.85	253680
macro avg	0.64	0.57	0.59	253680
weighted avg	0.81	0.85	0.82	253680

```
KNeighborsClassifier - Accuracy promedio (10-CV): 0.8470356354462314 ± 0.0015498727460175775
KNeighborsClassifier - AUC promedio (10-CV): 0.7134520514695797 ± 0.004474122160109038
```


Cuando se utiliza la tabla de variables seleccionadas automáticamente (X2), el rendimiento del modelo se mantiene muy similar en términos de **accuracy** 0,8470 y **precisión** para la clase 1 (0,40), pero se observa una ligera mejora en el **recall** de la clase 1 (0,20), lo que contribuye a un **f1-score** ligeramente más alto (0,26).

También mejora el **AUC** promedio, que sube a 0,7135, lo que indica que la reducción de variables no ha perjudicado el rendimiento general, sino que incluso podría haber ayudado al modelo a generalizar mejor.

3.2.5. GradientBoostingClassifier

Se implementa ahora el clasificador Gradient Boosting, repitiendo el proceso utilizado con los modelos anteriores. Este método de ensamble construye secuencialmente árboles de decisión que corrigen los errores cometidos por los anteriores, logrando así un modelo robusto y preciso. Se evaluará su rendimiento primero con la tabla original X.

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo GradientBoosting
gb_model = GradientBoostingClassifier()

# Obtener predicciones con validación cruzada
y = y.ravel()
y_pred = cross_val_predict(gb_model, X, y, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - GradientBoosting:")
print(confusion_matrix(y, y_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - GradientBoosting:")
print(classification_report(y, y_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(gb_model, X, y, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(gb_model, X, y, cv=cv, scoring='roc_auc')

print(f"\nGradientBoosting - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"GradientBoosting - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")
```

Matriz de Confusión - GradientBoosting:

```
[[214906  3428]
 [ 30826  4520]]
```

Reporte de Clasificación - GradientBoosting:

	precision	recall	f1-score	support
0	0.87	0.98	0.93	218334
1	0.57	0.13	0.21	35346
accuracy			0.86	253680
macro avg	0.72	0.56	0.57	253680
weighted avg	0.83	0.86	0.83	253680

GradientBoosting - Accuracy promedio (10-CV): 0.8649716177861875 ± 0.0007591140982065213
 GradientBoosting - AUC promedio (10-CV): 0.8075561432796994 ± 0.003968635232063453

El modelo alcanza una **accuracy** de 0,8650 y una **AUC** promedio de 0,8076, lo que refleja un buen equilibrio general. La clase mayoritaria (0) es identificada con alta **precisión** (0,87) y un **recall** de 0,98, lo que indica que casi todos los negativos son correctamente clasificados. No obstante, el rendimiento sobre la clase minoritaria (1) sigue siendo limitado: el **recall** es de 0,13 y el **f1-score** de 0,21, lo que pone de manifiesto una clara desventaja del modelo frente al desbalance de clases

A continuación, se evalúa su rendimiento con la tabla X2, donde se eliminaron variables según criterios automáticos.

```

from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo GradientBoosting
gb_model = GradientBoostingClassifier()

# Obtener predicciones con validación cruzada
y2 = y2.ravel()
y2_pred = cross_val_predict(gb_model, X2, y2, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - GradientBoosting:")
print(confusion_matrix(y2, y2_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - GradientBoosting:")
print(classification_report(y2, y2_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(gb_model, X2, y2, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(gb_model, X2, y2, cv=cv, scoring='roc_auc')

print(f"\nGradientBoosting - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"GradientBoosting - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")

```

Matriz de Confusión - GradientBoosting:

```
[[213985  4349]
 [ 29487  5859]]
```

Reporte de Clasificación - GradientBoosting:

	precision	recall	f1-score	support
0	0.88	0.98	0.93	218334
1	0.57	0.17	0.26	35346
accuracy			0.87	253680
macro avg	0.73	0.57	0.59	253680
weighted avg	0.84	0.87	0.83	253680

GradientBoosting - Accuracy promedio (10-CV): 0.8666193629769788 ± 0.0008519042989113918
 GradientBoosting - AUC promedio (10-CV): 0.8285347311930329 ± 0.0038672427423145613

Se puede observar que el rendimiento general mejora ligeramente. La **accuracy** asciende a 0,866 y la **AUC** promedio a 0,8285, mostrando una mejora notable en la capacidad discriminativa. Además, se observa una leve mejora en la detección de la clase 1, con un **recall** de 0,17 y un **f1-score** de 0,26.

3.2.6. XGBoost

En este apartado se aplica el modelo XGBoost, una versión optimizada y eficiente del algoritmo Gradient Boosting, muy utilizado en la práctica por su velocidad y rendimiento. Siguiendo la misma metodología que en los clasificadores previos, se entrenará el modelo con ambas versiones del conjunto de datos para observar cómo afecta la selección de variables a su eficacia.

Estudio sobre la tabla X:

```
import xgboost as xgb
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo XGBoost
xgb_model = xgb.XGBClassifier()

# Obtener predicciones con validación cruzada
y = y.values.ravel()
y_pred = cross_val_predict(xgb_model, X, y, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - XGBoost:")
print(confusion_matrix(y, y_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - XGBoost:")
print(classification_report(y, y_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(xgb_model, X, y, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(xgb_model, X, y, cv=cv, scoring='roc_auc')

print(f"\nXGBoost - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"\nXGBoost - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")
```

Matriz de Confusión - XGBoost:

```
[[214772  3562]
 [ 30841  4505]]
```

Reporte de Clasificación - XGBoost:

	precision	recall	f1-score	support
0	0.87	0.98	0.93	218334
1	0.56	0.13	0.21	35346
accuracy			0.86	253680
macro avg	0.72	0.56	0.57	253680
weighted avg	0.83	0.86	0.83	253680

```
\XGBoost - Accuracy promedio (10-CV): 0.8643842636392305 ± 0.0008869091411838741
XGBoost - AUC promedio (10-CV): 0.8052296832418172 ± 0.0038173558501274473
```

En la tabla X, el modelo XGBoost obtiene una **accuracy** del 0,8644 y un **AUC** promedio de 0,8052, lo que muestra un buen rendimiento general. La **precisión** de la clase 0 es de 0,87, y el **recall** es de 0,98, indicando que el modelo es muy eficaz para identificar los negativos. Sin embargo, la clase 1 sigue siendo problemática, con un **recall** de apenas 0,13 y un **f1-score** de 0,21, lo que revela una gran dificultad en la predicción de la clase positiva.

Con respecto a la tabla X2:

```
import xgboost as xgb
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo XGBoost
xgb_model = xgb.XGBClassifier()

# Obtener predicciones con validación cruzada
y2 = y2.ravel()
y2_pred = cross_val_predict(xgb_model, X2, y2, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - XGBoost:")
print(confusion_matrix(y2, y2_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - XGBoost:")
print(classification_report(y2, y2_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(xgb_model, X2, y2, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(xgb_model, X2, y2, cv=cv, scoring='roc_auc')

print(f"\nXGBoost - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"\nXGBoost - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")
```

Matriz de Confusión - XGBoost:

```
[[213723  4611]
 [ 29499  5847]]
```

Reporte de Clasificación - XGBoost:

	precision	recall	f1-score	support
0	0.88	0.98	0.93	218334
1	0.56	0.17	0.26	35346
accuracy			0.87	253680
macro avg	0.72	0.57	0.59	253680
weighted avg	0.83	0.87	0.83	253680

```
\XGBoost - Accuracy promedio (10-CV): 0.8655392620624408 ± 0.0011503542394809714
XGBoost - AUC promedio (10-CV): 0.8256607259927025 ± 0.0036184312202205125
```

El rendimiento mejora ligeramente. La **accuracy** sube a 0,8655 y el **AUC** promedio aumenta a 0,8257, lo que refleja una mejora en la capacidad de discriminación entre las dos clases. La detección de la clase 1 también mejora con un **recall** de 17 % y un **f1-score** de 0,26, aunque sigue siendo insuficiente para un modelo ideal.

3.2.7. Naive Bayes

Por último, se utiliza el clasificador Naive Bayes, un modelo probabilístico basado en el teorema de Bayes con la suposición de independencia entre variables. Aunque es un método sencillo, suele ofrecer buenos resultados en clasificación binaria, especialmente en contextos con grandes volúmenes de datos. Como en los casos anteriores, se probará con ambas tablas de variables explicativas.

Primero se aplica el clasificador a la tabla empelada en el nivel básico.

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo Naive Bayes (GaussianNB)
nb_model = GaussianNB()

# Obtener predicciones con validación cruzada
y = y.ravel()
y_pred = cross_val_predict(nb_model, X, y, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - Naive Bayes (GaussianNB):")
print(confusion_matrix(y, y_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - Naive Bayes (GaussianNB):")
print(classification_report(y, y_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(nb_model, X, y, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(nb_model, X, y, cv=cv, scoring='roc_auc')

print(f"\nNaive Bayes (GaussianNB) - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Naive Bayes (GaussianNB) - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")
```

```
Matriz de Confusión - Naive Bayes (GaussianNB):
[[187348  30986]
 [ 19050 16296]]
```

```
Reporte de Clasificación - Naive Bayes (GaussianNB):
              precision    recall  f1-score   support

     0       0.91         0.86         0.88       218334
     1       0.34         0.46         0.39        35346

 accuracy                   0.80       253680
 macro avg              0.63         0.66         0.64       253680
 weighted avg          0.83         0.80         0.81       253680
```

```
Naive Bayes (GaussianNB) - Accuracy promedio (10-CV): 0.8027593818984547 ± 0.0019262665473447399
Naive Bayes (GaussianNB) - AUC promedio (10-CV): 0.7758720981525322 ± 0.004692810288287942
```

El modelo Naive Bayes (GaussianNB) alcanza una **accuracy** del 0,8028 y un **AUC** promedio de 0,7759. La **precisión** de la clase 0 es bastante alta (0,91), y el **recall** es de 0,86, indicando que el modelo es bastante bueno en identificar la clase mayoritaria (0). Sin embargo, la detección de la clase minoritaria (1) sigue siendo limitada, con un **recall** de 0,46 y un **f1-score** de 0,39, lo que indica que, aunque hay una mejora en la predicción de la clase positiva, aún hay un margen importante de mejora.

Después se prueba el mismo clasificador sobre la segunda tabla (X2).

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo Naive Bayes (GaussianNB)
nb_model = GaussianNB()

# Obtener predicciones con validación cruzada
y2 = y2.ravel()
y2_pred = cross_val_predict(nb_model, X2, y2, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - Naive Bayes (GaussianNB):")
print(confusion_matrix(y2, y2_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - Naive Bayes (GaussianNB):")
print(classification_report(y2, y2_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(nb_model, X2, y2, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(nb_model, X2, y2, cv=cv, scoring='roc_auc')

print(f"\nNaive Bayes (GaussianNB) - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Naive Bayes (GaussianNB) - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")
```

```
Matriz de Confusión - Naive Bayes (GaussianNB):
[[181945  36389]
 [ 16942 18404]]
```

```
Reporte de Clasificación - Naive Bayes (GaussianNB):
              precision    recall  f1-score   support

     0       0.91         0.83         0.87         218334
     1       0.34         0.52         0.41          35346

 accuracy                   0.79         253680
 macro avg       0.63         0.68         0.64         253680
 weighted avg    0.83         0.79         0.81         253680
```

```
Naive Bayes (GaussianNB) - Accuracy promedio (10-CV): 0.7897705771050142 ± 0.0020415810983931593
Naive Bayes (GaussianNB) - AUC promedio (10-CV): 0.7885649555933139 ± 0.00379990708383032
```

En este caso, el rendimiento disminuye ligeramente, con una **accuracy** del 0,7898 y un **AUC** promedio de 0,7886. La **precisión** de la clase 0 sigue siendo alta (0,91), pero el **recall** bajo (0,83), lo que implica que el modelo identifica menos instancias de la clase negativa. La clase 1 mejora su **recall** al 0,52 y el **f1-score** sube a 0,41, lo que refleja una mejora modesta respecto a la tabla X, aunque sigue siendo un área de oportunidad para el modelo.

La disminución en el rendimiento al usar la tabla X2 indica que la selección de variables en este caso no favorece mucho a este modelo, lo que sugiere que otras técnicas podrían ser más eficaces para optimizarlo.

3.2.8. Comparación de los clasificadores medios

	Accuracy (10-CV)	AUC (10-CV)	Precision		Recall		F1-Score	
			Clase 0	Clase 1	Clase 0	Clase 1	Clase 0	Clase 1
Logistic Regression (X)	0,8624	0,7998	0,87	0,53	0,98	0,12	0,92	0,19
Logistic Regression (X2)	0,8634	0,8204	0,88	0,53	0,98	0,15	0,92	0,24
Random Forest (X)	0,8415	0,7079	0,88	0,37	0,94	0,20	0,91	0,26
Random Forest (X2)	0,8497	0,7472	0,88	0,41	0,96	0,19	0,92	0,26
Decision Tree (X)	0,8371	0,6181	0,88	0,35	0,94	0,20	0,91	0,25
Decision Tree (X2)	0,8035	0,5979	0,89	0,30	0,88	0,31	0,89	0,30
KNeighbors (X)	0,8474	0,6971	0,88	0,40	0,95	0,18	0,92	0,25
KNeighbors (X2)	0,8470	0,7135	0,88	0,40	0,95	0,20	0,91	0,26
Gradient Boosting (X)	0,8650	0,8076	0,87	0,57	0,98	0,13	0,93	0,21
Gradient Boosting (X2)	0,8666	0,8285	0,88	0,57	0,98	0,17	0,93	0,26
XGBoost (X)	0,8644	0,8052	0,87	0,56	0,98	0,13	0,93	0,21
XGBoost (X2)	0,8655	0,8257	0,88	0,56	0,98	0,17	0,93	0,26
Naive Bayes (X)	0,8028	0,7759	0,91	0,34	0,86	0,46	0,88	0,39
Naive Bayes (X2)	0,7898	0,7886	0,91	0,34	0,83	0,52	0,87	0,41

En esta sección, hemos evaluado distintos clasificadores sobre dos conjuntos de datos: X y X2, obteniendo métricas clave como Accuracy (10-CV), AUC (10-CV), así como los resultados de las métricas de precision, recall y F1-score para las clases 0 y 1. Estas métricas nos permiten tener una visión clara del rendimiento de cada modelo tanto en términos generales como en el desempeño sobre ambas clases.

Los resultados muestran que Gradient Boosting (X2) y XGBoost (X2) son los clasificadores más robustos, con el mejor desempeño en términos de accuracy y AUC. Sin embargo, Logistic Regression y KNeighbors también presentan buenos resultados, con precision elevada para la clase mayoritaria (0), pero sin un desempeño destacado sobre la clase minoritaria (1). Por otro lado, Naive Bayes tiene una precisión bastante alta para la clase 0, pero un desempeño deficiente con la clase 1.

El uso de X2 (que incluye variables seleccionadas automáticamente) mejora los resultados en general, especialmente en AUC y en la detección de la clase minoritaria, lo que refuerza la idea de que una buena selección de variables puede optimizar significativamente el rendimiento de los modelos. Por ello, X2 va a ser la tabla a estudiar en el resto del proyecto.

3.3. Ajuste de los parámetros

Una vez entrenados los clasificadores del nivel medio, el siguiente paso natural para mejorar su rendimiento consiste en ajustar sus parámetros internos. Para llevar a cabo esta tarea, se ha utilizado la función **GridSearchCV()** de la biblioteca scikit-learn, una herramienta potente que permite realizar una búsqueda exhaustiva sobre un espacio definido de combinaciones de hiperparámetros. Esta búsqueda se realiza mediante una validación cruzada, lo que garantiza que los resultados no dependan únicamente de una partición específica de los datos. Mediante esta función se especifica un modelo base y un diccionario de parámetros con los valores a explorar. (Ref 21)

Este procedimiento se ha aplicado a todos los clasificadores medios para refinar su rendimiento y obtener una configuración más adecuada al problema. Para programar un código funcional se ha tomado de referencia los ejemplos de la referencia 22.

3.3.1. LogisticRegression

Para el ajuste de LogisticRegression, se definió una búsqueda con GridSearchCV() para explorar diversas combinaciones de hiperparámetros relevantes. Se evaluaron diferentes valores del parámetro de regularización C, así como dos tipos de penalización (l1, l2) y dos solvers de optimización (liblinear y saga). Además, se ajustaron el número máximo de iteraciones (max_iter) y la tolerancia de convergencia (tol) para asegurar una convergencia adecuada del modelo. (Ref 12)

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, cross_val_score
import numpy as np

# Modelo base
logreg = LogisticRegression(random_state=42)

# Espacio de hiperparámetros con valores discretos (válido para GridSearch)
param_grid = {
    'C': [0.005, 0.01, 0.1, 1, 10],
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear', 'saga'],
    'max_iter': [500, 1000],
    'tol': [1e-6, 1e-5]
}

# Regularización
# Penalización
# Optimizador
# Iteraciones
# Tolerancia de convergencia

# Configurar GridSearchCV
grid_search_logreg = GridSearchCV(
    estimator=logreg,
    param_grid=param_grid,
    scoring='roc_auc',
    n_jobs=-1,
    cv=5,
    verbose=2
)

# Entrenar GridSearchCV
y2_train = y2_train.values.ravel()
grid_search_logreg.fit(X2_train, y2_train)

# Mostrar los mejores hiperparámetros
print("Mejores hiperparámetros encontrados para Regresión Logística:")
print(grid_search_logreg.best_params_)

# Modelo final con los mejores hiperparámetros
best_logreg = grid_search_logreg.best_estimator_

# Evaluación con validación cruzada (10-CV)
auc_scores_logreg = cross_val_score(best_logreg, X2, y2, cv=10, scoring='roc_auc')
print(f"AUC promedio del modelo final (10-CV): {np.mean(auc_scores_logreg)} ± {np.std(auc_scores_logreg)}")
```



```
Fitting 5 folds for each of 80 candidates, totalling 400 fits
Mejores hiperparámetros encontrados para Regresión Logística:
{'C': 0.005, 'max_iter': 500, 'penalty': 'l2', 'solver': 'saga', 'tol': 1e-06}
AUC promedio del modelo final (10-CV): 0.8204344674819662 ± 0.006790321538518012
```

Tras evaluar 400 combinaciones de parámetros, el mejor modelo se obtuvo con una regularización C muy baja (0,005), usando penalización l2 y el solver saga. El modelo alcanzó un AUC promedio de 0,8204.

3.3.2. RandomForestClassifier

En el caso de RandomForestClassifier, se utilizó GridSearchCV() para ajustar parámetros que controlan tanto la complejidad del modelo como su estabilidad. Se exploraron distintos valores para el número de árboles (n_estimators), la profundidad máxima (max_depth), el número mínimo de muestras para dividir un nodo (min_samples_split) y el número mínimo de muestras en una hoja (min_samples_leaf). También se incluyó el método para seleccionar características (max_features = 'log2') y la estrategia de *bootstrap* para asegurar una diversidad adecuada entre los árboles. (Ref 14)

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, cross_val_score
import numpy as np

# Definir el modelo base
rf = RandomForestClassifier(random_state=42)

# Espacio de hiperparámetros (valores concretos, no distribuciones)
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [10, 20],
    'min_samples_split': [10, 15],
    'min_samples_leaf': [2, 4],
    'max_features': ['log2'],
    'bootstrap': [True]
}

# Configurar GridSearchCV
grid_search_rf = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    scoring='roc_auc',
    n_jobs=-1,
    cv=5,
    verbose=2
)

# Ejecutar búsqueda
# y2_train = y2_train.values.ravel()
grid_search_rf.fit(X2_train, y2_train)

# Mostrar mejores hiperparámetros
print("Mejores hiperparámetros encontrados para Random Forest:")
print(grid_search_rf.best_params_)

# Modelo final y evaluación
best_rf = grid_search_rf.best_estimator_
auc_scores_rf = cross_val_score(best_rf, X2, y2, cv=10, scoring='roc_auc')
print(f"AUC promedio del Random Forest final (10-CV): {np.mean(auc_scores_rf)} ± {np.std(auc_scores_rf)}")
```

```
Fitting 5 folds for each of 16 candidates, totalling 80 fits
Mejores hiperparámetros encontrados para Random Forest:
{'bootstrap': True, 'max_depth': 10, 'max_features': 'log2', 'min_samples_leaf': 4, 'min_samples_split': 15, 'n_estimators': 200}
AUC promedio del Random Forest final (10-CV): 0.8256816128693407 ± 0.004304419725053121
```

3.3.3. DecisionTreeClassifier

Para DecisionTreeClassifier, se llevó a cabo un ajuste de parámetros utilizando GridSearchCV() centrado en controlar la complejidad del árbol y su capacidad para generalizar. Se incluyeron combinaciones de criterios de partición (gini y entropy), profundidades máximas (max_depth), mínimos de muestras para dividir (min_samples_split) y mínimos en las hojas (min_samples_leaf). También se variaron las estrategias de selección de características (max_features = 'sqrt' y 'log2'). (Ref 23)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, cross_val_score
import numpy as np

# Modelo base
dt = DecisionTreeClassifier(random_state=42)

# Definición del espacio de hiperparámetros con valores concretos
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [5, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2']
}

# función de medida de impureza
# profundidad máxima del árbol
# mínimo de muestras para dividir
# mínimo de muestras en una hoja
# número de características a considerar

# Configuración de GridSearchCV
grid_search_dt = GridSearchCV(
    estimator=dt,
    param_grid=param_grid,
    scoring='roc_auc',
    n_jobs=-1,
    cv=5,
    verbose=2
)

# Entrenamiento con el conjunto de entrenamiento
#y2_train = y2_train.values.ravel()
grid_search_dt.fit(X2_train, y2_train)

# Mejores hiperparámetros
print("Mejores hiperparámetros encontrados para Decision Tree:")
print(grid_search_dt.best_params_)

# Modelo final
best_dt = grid_search_dt.best_estimator_

# Evaluación con validación cruzada (10-CV)
auc_scores_dt = cross_val_score(best_dt, X2, y2, cv=10, scoring='roc_auc')
print(f"AUC promedio del Decision Tree final (10-CV): {np.mean(auc_scores_dt)} ± {np.std(auc_scores_dt)}")
```

```
Fitting 5 folds for each of 72 candidates, totalling 360 fits
Mejores hiperparámetros encontrados para Decision Tree:
{'criterion': 'entropy', 'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 5}
AUC promedio del Decision Tree final (10-CV): 0.8036759731140272 ± 0.005612601030673793
```

El ajuste fino del árbol de decisión, explorando 72 combinaciones, mostró que el uso del criterio entropy y una profundidad máxima de 10 mejora el rendimiento.

3.3.4. KNeighborsClassifier

Para el modelo KNeighborsClassifier, el proceso de ajuste de parámetros se llevó a cabo mediante GridSearchCV() con el objetivo de encontrar el número óptimo de vecinos (n_neighbors), así como la mejor estrategia de ponderación, ya sea uniforme o basada en distancia. También se evaluó el uso de la métrica de Minkowski junto con su parámetro p, permitiendo alternar entre la distancia de Manhattan (p=1) y la euclídea (p=2). (Ref 24)

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV, cross_val_score
import numpy as np

# Modelo base
knn = KNeighborsClassifier()

# Espacio de hiperparámetros
param_grid = {
    'n_neighbors': [5, 7, 9],
    'weights': ['uniform', 'distance'],
    'metric': ['minkowski'],
    'p': [1, 2]
}

# número de vecinos
# ponderación de los vecinos
# métrica de distancia
# parámetro para Minkowski

# Configurar GridSearchCV
grid_search_knn = GridSearchCV(
    estimator=knn,
    param_grid=param_grid,
    scoring='roc_auc',
    n_jobs=-1,
    cv=5,
    verbose=2
)

# Ejecutar búsqueda
#y2_train = y2_train.values.ravel()
grid_search_knn.fit(X2_train, y2_train.ravel())

# Mejores hiperparámetros
print("Mejores hiperparámetros encontrados para KNN:")
print(grid_search_knn.best_params_)

# Modelo final con los mejores parámetros
best_knn = grid_search_knn.best_estimator_

# Evaluar con validación cruzada (10-CV)
auc_scores_knn = cross_val_score(best_knn, X2, y2.ravel(), cv=10, scoring='roc_auc')
print(f"AUC promedio del KNN final (10-CV): {np.mean(auc_scores_knn)} ± {np.std(auc_scores_knn)}")

```

Fitting 5 folds for each of 12 candidates, totalling 60 fits

Mejores hiperparámetros encontrados para KNN:

```
{'metric': 'minkowski', 'n_neighbors': 9, 'p': 1, 'weights': 'uniform'}
```

```
AUC promedio del KNN final (10-CV): 0.7540034917151222 ± 0.004086358519536594
```

Probando 5 pliegues, cada uno de 12 combinaciones, el mejor KNN fue con 9 vecinos, métrica de Minkowski (p=1, equivalente a Manhattan) y ponderación uniforme. Sin embargo, el AUC obtenido fue solo 0,7540, siendo el más bajo entre todos los modelos ajustados hasta el momento.

3.3.5. GradientBoostingClassifier

En el caso del modelo GradientBoostingClassifier, se aplicó GridSearchCV() para afinar una variedad de hiperparámetros críticos para el rendimiento del modelo ensamblado. Se ajustaron el número de árboles (n_estimators), la tasa de aprendizaje (learning_rate), la profundidad máxima de los árboles (max_depth), y los parámetros relacionados con la división de nodos (min_samples_split, min_samples_leaf). También se consideraron configuraciones para el muestreo (subsample) y el número de características usadas (max_features). (Ref 25)

```

from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import GridSearchCV, cross_val_score
import numpy as np

# Modelo base
gb = GradientBoostingClassifier(random_state=42)

# Espacio de hiperparámetros
param_grid = {
    'n_estimators': [100],
    'learning_rate': [0.05, 0.1],
    'max_depth': [3, 5],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'subsample': [0.8],
    'max_features': ['sqrt', None]
}

# Configurar GridSearchCV
grid_search_gb = GridSearchCV(
    estimator=gb,
    param_grid=param_grid,
    scoring='roc_auc',
    n_jobs=-1,
    cv=5,
    verbose=2
)

# Entrenamiento
#y2_train = y2_train.ravel()
grid_search_gb.fit(X2_train, y2_train)

# Mejores hiperparámetros
print("Mejores hiperparámetros encontrados para Gradient Boosting:")
print(grid_search_gb.best_params_)

# Modelo final
best_gb = grid_search_gb.best_estimator_

# Evaluación final con validación cruzada 10-CV
auc_scores_gb = cross_val_score(best_gb, X2, y2.ravel(), cv=10, scoring='roc_auc')
print(f"AUC promedio del Gradient Boosting final (10-CV): {np.mean(auc_scores_gb)} ± {np.std(auc_scores_gb)}")

```

```

Fitting 5 folds for each of 32 candidates, totalling 160 fits
Mejores hiperparámetros encontrados para Gradient Boosting:
{'learning_rate': 0.1, 'max_depth': 5, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 100, 'subsample': 0.8}
AUC promedio del Gradient Boosting final (10-CV): 0.8287746154038199 ± 0.00463488666530486

```

Este modelo probó 5 pliegues, cada uno de 32 combinaciones, seleccionando finalmente una configuración con `learning_rate=0,1`, profundidad 5 y `subsample=0,8`. El AUC final fue 0.8288, uno de los más altos del análisis. Esto demuestra que el corregir los errores del modelo anterior resulta muy eficaz para esta tarea de clasificación.

3.3.6. XGBoost

Para XGBoost, se definió una búsqueda exhaustiva de hiperparámetros mediante `GridSearchCV()` centrada en la capacidad del modelo para generalizar y manejar datos complejos. Se exploraron combinaciones de número de estimadores (`n_estimators`), tasas de aprendizaje (`learning_rate`), profundidad de los árboles (`max_depth`), peso mínimo de hijos (`min_child_weight`), así como los porcentajes de submuestreo de filas (`subsample`) y columnas (`colsample_bytree`).

Para escribir el siguiente código se han consultado las referencias 7 y 26.

```

from xgboost import XGBClassifier
from sklearn.model_selection import GridSearchCV, cross_val_score
import numpy as np

# Modelo base
xgb = XGBClassifier(random_state=42)

# Espacio de hiperparámetros
param_grid = {
    'n_estimators': [100],
    'learning_rate': [0.05, 0.1],
    'max_depth': [3, 5],
    'min_child_weight': [1, 2],
    'subsample': [0.8],
    'colsample_bytree': [0.8, 1.0]
}

# Configurar GridSearchCV
grid_search_xgb = GridSearchCV(
    estimator=xgb,
    param_grid=param_grid,
    scoring='roc_auc',
    n_jobs=-1,
    cv=5,
    verbose=2
)

# Entrenamiento
# y2_train = y2_train.ravel()
grid_search_xgb.fit(X2_train, y2_train)

# Mejores hiperparámetros
print("Mejores hiperparámetros encontrados para XGBoost:")
print(grid_search_xgb.best_params_)

# Modelo final
best_xgb = grid_search_xgb.best_estimator_

# Evaluación final con validación cruzada 10-CV
auc_scores_xgb = cross_val_score(best_xgb, X2, y2.ravel(), cv=10, scoring='roc_auc')
print(f"AUC promedio del XGBoost final (10-CV): {np.mean(auc_scores_xgb)} ± {np.std(auc_scores_xgb)}")

```

Fitting 5 folds for each of 16 candidates, totalling 80 fits

Mejores hiperparámetros encontrados para XGBoost:

{'colsample_bytree': 0.8, 'learning_rate': 0.1, 'max_depth': 5, 'min_child_weight': 2, 'n_estimators': 100, 'subsample': 0.8}

AUC promedio del XGBoost final (10-CV): 0.8289206729005139 ± 0.004608657323714063

Con una estructura similar al Gradient Boosting, pero con mejoras en velocidad y regularización, XGBoost alcanzó un AUC de 0,8289, el más alto del conjunto.

3.3.7. Naive Bayes

Para GaussianNB, modelo basado en probabilidad, el ajuste de parámetros fue más sencillo, centrándose únicamente en el hiperparámetro `var_smoothing`, el cual suaviza las varianzas calculadas para evitar divisiones por cero o problemas numéricos. (Ref 27)

```

from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import GridSearchCV, cross_val_score
import numpy as np

# Modelo base
nb = GaussianNB()

# Espacio de hiperparámetros
param_grid = {
    'var_smoothing': [1e-9, 1e-8, 1e-7]
}

```

```

# Configurar GridSearchCV
grid_search_nb = GridSearchCV(
    estimator=nb,
    param_grid=param_grid,
    scoring='roc_auc',
    n_jobs=-1,
    cv=5,
    verbose=2
)

# Entrenamiento
# y2_train = y2_train.ravel()
grid_search_nb.fit(X2_train, y2_train)

# Mejores hiperparámetros
print("Mejores hiperparámetros encontrados para Naive Bayes:")
print(grid_search_nb.best_params_)

# Modelo final
best_nb = grid_search_nb.best_estimator_

# Evaluación final con validación cruzada 10-CV
auc_scores_nb = cross_val_score(best_nb, X2, y2.ravel(), cv=10, scoring='roc_auc')
print(f"AUC promedio del Naive Bayes final (10-CV): {np.mean(auc_scores_nb)} ± {np.std(auc_scores_nb)}")

```

Fitting 5 folds for each of 3 candidates, totalling 15 fits

Mejores hiperparámetros encontrados para Naive Bayes:

{'var_smoothing': 1e-07}

AUC promedio del Naive Bayes final (10-CV): 0.7885296651676079 ± 0.006031554421890719

Aunque solo se probaron 3 valores para var_smoothing, el mejor modelo alcanzó un AUC de 0,7885, lo cual es destacable para un clasificador tan simple y rápido. Su rendimiento fue inferior al de los métodos más complejos.

3.3.8. Comparación de los clasificadores medios con hiperparámetros

En la siguiente tabla se presentan los valores del AUC promedio obtenidos mediante validación cruzada (10-CV) sobre el conjunto de datos X2. Previamente, estos modelos fueron evaluados utilizando sus hiperparámetros por defecto. En esta etapa, se han recalculado los valores del AUC tras realizar un ajuste de hiperparámetros utilizando la técnica de GridSearchCV(), lo que ha permitido optimizar el rendimiento de cada clasificador.

AUC 10-CV EN X2	Sin ajuste	Con ajuste
Logistic Regression	0,8204	0,8204
Random Forest	0,7472	0,8257
Decision Tree	0,5979	0,8037
KNeighbors	0,7135	0,7540
Gradient Boosting	0,8285	0,8288
XGBoost	0,8257	0,8289
Naive Bayes	0,7886	0,7885

En conjunto, se evidencia que los modelos más complejos y con mayor número de hiperparámetros (como Random Forest o Decision Tree) tienden a beneficiarse más del proceso de ajuste, mientras que modelos más simples o ya bien optimizados (como Logistic Regression o Naive Bayes) se mantienen constantes.

3.4. Clasificadores sensibles al coste

En esta sección se emplearán clasificadores sensibles al coste, una estrategia alternativa al oversampling y undersampling para abordar el problema de desbalance de clases. En lugar de modificar la distribución de los datos, esta técnica consiste en ajustar el parámetro **class_weight** en ciertos algoritmos de clasificación. Este parámetro permite dar mayor peso a la clase minoritaria durante el entrenamiento, penalizando más los errores cometidos sobre dicha clase y, en consecuencia, mejorando el rendimiento del modelo sobre ella. (Ref 28)

No todos los clasificadores analizados anteriormente permiten el uso del parámetro `class_weight`. Por ello, el estudio en esta parte se limitará a aquellos modelos que sí admiten esta funcionalidad, concretamente: Logistic Regression, Random Forest y Decision Tree.

Además, al igual que se realizó en el nivel básico con las técnicas de oversampling y undersampling, se evaluará el efecto del ajuste de `class_weight` tanto sin como con ajuste de hiperparámetros. El objetivo de este enfoque es observar si el ajuste del coste de clasificación puede mejorar el rendimiento del modelo sobre la clase minoritaria sin alterar la distribución original de los datos, y si este efecto se ve potenciado al combinarlo con una adecuada selección de hiperparámetros.

3.4.1. LogisticRegression

En una primera instancia, se ha evaluado el modelo sin ajustar el resto de hiperparámetros, manteniéndolos en sus valores por defecto. Posteriormente, se ha realizado un segundo experimento en el que se ha combinado `class_weight='balanced'` con otros hiperparámetros recomendados por GridSearchCV en el paso anterior, para observar si esta combinación mejora el rendimiento del modelo en comparación con el uso aislado del ajuste de coste o la optimización estándar. (Ref 29)

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.metrics import classification_report, f1_score, accuracy_score, confusion_matrix
from sklearn.model_selection import cross_val_predict

# Crear el modelo de regresión logística con class weight='balanced'
logreg = LogisticRegression(class_weight='balanced', max_iter=1000)

# Realizar la validación cruzada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Realizamos las predicciones para cada pliegue de validación cruzada
y2_pred = cross_val_predict(logreg, X2, y2, cv=cv)
```

```

# Matriz de confusión
conf_matrix = confusion_matrix(y2, y2_pred)
print("\nMatriz de Confusión:")
print(conf_matrix)

# Mostrar el reporte de clasificación completo
print("Reporte de clasificación para el modelo con class_weight='balanced':")
print(classification_report(y2, y2_pred, zero_division=0))

# Evaluación con 10-CV (Accuracy)
acc_cv = cross_val_score(logreg, X2, y2, cv=cv, scoring='accuracy')
print(f"Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")

# AUC promedio utilizando 10-CV
auc_cv = cross_val_score(logreg, X2, y2, cv=cv, scoring='roc_auc')
print(f"AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")

```

A continuación, como ha indicado anteriormente, se presenta el código de la combinación entre `class_weight='balanced'` y los mejores hiperparámetros para este estudio en regresión logística.

```

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.metrics import classification_report, f1_score, accuracy_score, confusion_matrix
from sklearn.model_selection import cross_val_predict

# Crear el modelo de regresión logística con class_weight='balanced'
logreg = LogisticRegression(
    class_weight='balanced',
    C=0.005,
    max_iter=500,
    penalty='l2',
    solver='saga',
    tol=1e-6,
    random_state=42
)

# Realizar la validación cruzada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

y2 = y2.values.ravel()
# Realizamos las predicciones para cada pliegue de validación cruzada
y2_pred = cross_val_predict(logreg, X2, y2, cv=cv)

# Matriz de confusión
conf_matrix = confusion_matrix(y2, y2_pred)
print("\nMatriz de Confusión:")
print(conf_matrix)

# Mostrar el reporte de clasificación completo
print("Reporte de clasificación para el modelo con class_weight='balanced':")
print(classification_report(y2, y2_pred, zero_division=0))

# Evaluación con 10-CV (Accuracy)
acc_cv = cross_val_score(logreg, X2, y2, cv=cv, scoring='accuracy')
print(f"Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")

# AUC promedio utilizando 10-CV
auc_cv = cross_val_score(logreg, X2, y2, cv=cv, scoring='roc_auc')
print(f"AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")

```


Los resultados obtenidos para la regresión logística utilizando un clasificador sensible al coste se presentan a continuación, diferenciando entre el modelo sin ajuste de hiperparámetros y el modelo con ajuste.

```
Matriz de Confusión:
[[158508  59826]
 [ 8372 26974]]
Reporte de clasificación para el modelo con class_weight='balanced':
```

	precision	recall	f1-score	support
0	0.95	0.73	0.82	218334
1	0.31	0.76	0.44	35346
accuracy			0.73	253680
macro avg	0.63	0.74	0.63	253680
weighted avg	0.86	0.73	0.77	253680

```
Accuracy promedio (10-CV): 0.7311652475559759 ± 0.002859646175846172
AUC promedio (10-CV): 0.8209515391539581 ± 0.004003396824424076
```

```
Matriz de Confusión:
[[158517  59817]
 [ 8370 26976]]
Reporte de clasificación para el modelo con class_weight='balanced':
```

	precision	recall	f1-score	support
0	0.95	0.73	0.82	218334
1	0.31	0.76	0.44	35346
accuracy			0.73	253680
macro avg	0.63	0.74	0.63	253680
weighted avg	0.86	0.73	0.77	253680

```
Accuracy promedio (10-CV): 0.7312086092715232 ± 0.002774547794751039
AUC promedio (10-CV): 0.8209570347914739 ± 0.00397841925645709
```

La incorporación del parámetro `class_weight='balanced'` en la regresión logística ha tenido un efecto notable en la capacidad del modelo para identificar correctamente la clase minoritaria. En ambos escenarios (con y sin ajuste de hiperparámetros) el modelo alcanza un **recall** del 0,76 para la clase 1, una mejora sustancial en comparación con las configuraciones previas que no aplicaban balanceo de clases, donde este valor era considerablemente inferior.

No obstante, esta ganancia en sensibilidad viene acompañada de una disminución en la **precisión** (0,31), lo que sugiere un incremento en los falsos positivos. Además, al comparar ambos modelos, se evidencia que el ajuste de hiperparámetros no aporta mejoras significativas, ya que los valores de las métricas principales (accuracy, recall, F1-score y AUC) se mantienen prácticamente los mismos.

Por último, es importante destacar que, pese a la mejora en el equilibrio de clases, el valor del **AUC** se mantiene constante con respecto al obtenido en el estudio previo sin emplear `class_weight`.

3.4.2. RandomForestClassifier

En el caso del clasificador Random Forest, también se ha aplicado `class_weight='balanced'` para penalizar más los errores sobre la clase minoritaria. Al igual que con la regresión logística, se ha ejecutado una primera versión del modelo sin modificar los demás hiperparámetros, y una segunda versión en la que se ha llevado a cabo un proceso de ajuste conjunto de hiperparámetros. Se presentan ambos códigos a continuación:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold, cross_val_predict
from sklearn.metrics import classification_report, f1_score, accuracy_score, confusion_matrix
import numpy as np

# Crear el modelo Random Forest con class_weight='balanced'
rf = RandomForestClassifier(class_weight='balanced', random_state=42)

# Validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Predicciones usando validación cruzada
y2_pred_rf = cross_val_predict(rf, X2, y2, cv=cv)

# Matriz de confusión
conf_matrix_rf = confusion_matrix(y2, y2_pred_rf)
print("\nMatriz de Confusión (Random Forest):")
print(conf_matrix_rf)

# Reporte de clasificación
print("Reporte de clasificación para Random Forest con class_weight='balanced':")
print(classification_report(y2, y2_pred_rf, zero_division=0))

# Accuracy promedio
acc_cv_rf = cross_val_score(rf, X2, y2, cv=cv, scoring='accuracy')
print(f"Accuracy promedio del Random Forest (10-CV): {acc_cv_rf.mean()} ± {acc_cv_rf.std()}")

# AUC promedio
auc_cv_rf = cross_val_score(rf, X2, y2, cv=cv, scoring='roc_auc')
print(f"AUC promedio del Random Forest (10-CV): {auc_cv_rf.mean()} ± {auc_cv_rf.std()}")
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold, cross_val_predict
from sklearn.metrics import classification_report, f1_score, accuracy_score, confusion_matrix
import numpy as np

# Crear el modelo Random Forest con class_weight='balanced'
rf = RandomForestClassifier(
    bootstrap=True,
    max_depth=11,
    max_features='log2',
    min_samples_leaf=4,
    min_samples_split=15,
    n_estimators=100,
    random_state=42,
    class_weight='balanced'
)

# Validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Predicciones usando validación cruzada
y2_pred_rf = cross_val_predict(rf, X2, y2, cv=cv)
```

```

# Matriz de confusión
conf_matrix_rf = confusion_matrix(y2, y2_pred_rf)
print("\nMatriz de Confusión (Random Forest):")
print(conf_matrix_rf)

# Reporte de clasificación
print("Reporte de clasificación para Random Forest con class_weight='balanced':")
print(classification_report(y2, y2_pred_rf, zero_division=0))

# Accuracy promedio
acc_cv_rf = cross_val_score(rf, X2, y2, cv=cv, scoring='accuracy')
print(f"Accuracy promedio del Random Forest (10-CV): {acc_cv_rf.mean()} ± {acc_cv_rf.std()}")

# AUC promedio
auc_cv_rf = cross_val_score(rf, X2, y2, cv=cv, scoring='roc_auc')
print(f"AUC promedio del Random Forest (10-CV): {auc_cv_rf.mean()} ± {auc_cv_rf.std()}")

```

Los resultados para el clasificador Random Forest con sensibilidad al coste se presentan en dos fases: primero sin ajuste de hiperparámetros y luego incorporando el ajuste.

Matriz de Confusión (Random Forest):

```
[[207619 10715]
 [ 28385 6961]]
```

Reporte de clasificación para Random Forest con class_weight='balanced':

	precision	recall	f1-score	support
0	0.88	0.95	0.91	218334
1	0.39	0.20	0.26	35346
accuracy			0.85	253680
macro avg	0.64	0.57	0.59	253680
weighted avg	0.81	0.85	0.82	253680

Accuracy promedio del Random Forest (10-CV): 0.8458688111005991 ± 0.0016282786925527883
AUC promedio del Random Forest (10-CV): 0.777241311304216 ± 0.0032761736686786483

Matriz de Confusión (Random Forest):

```
[[159326 59008]
 [ 8253 27093]]
```

Reporte de clasificación para Random Forest con class_weight='balanced':

	precision	recall	f1-score	support
0	0.95	0.73	0.83	218334
1	0.31	0.77	0.45	35346
accuracy			0.73	253680
macro avg	0.63	0.75	0.64	253680
weighted avg	0.86	0.73	0.77	253680

Accuracy promedio del Random Forest (10-CV): 0.7348588773257647 ± 0.0028950009455189487
AUC promedio del Random Forest (10-CV): 0.8264104187675061 ± 0.004082260969679521

En este caso, no se observan diferencias significativas entre el modelo Random Forest con class_weight='balanced' y sin dicho parámetro, siempre que no se haya realizado ajuste de hiperparámetros. Esto indica que el uso de class_weight aporta una mejora sustancial respecto al modelo base, pero no alcanza el nivel de mejora que se obtiene mediante el ajuste de hiperparámetros. No obstante, la combinación de ambos enfoques (ajuste de hiperparámetros y balanceo de clases) es la que ofrece el mejor rendimiento general.

3.4.3. DecisionTreeClassifier

Para el modelo Decision Tree, se ha seguido la misma metodología: se ha utilizado `class_weight='balanced'`. Primero, se ha probado el árbol de decisión con los parámetros por defecto y únicamente el ajuste de pesos activado. En segundo lugar, se ha realizado una búsqueda de hiperparámetros incorporando `class_weight` junto con otros parámetros clave del algoritmo.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo Decision Tree
decision_tree = DecisionTreeClassifier(class_weight='balanced', random_state=42)

# Obtener predicciones con validación cruzada
y2_pred = cross_val_predict(decision_tree, X2, y2, cv=cv)

# Matriz de confusión
print("\nMatriz de Confusión - Decision Tree:")
print(confusion_matrix(y2, y2_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - Decision Tree:")
print(classification_report(y2, y2_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(decision_tree, X2, y2, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(decision_tree, X2, y2, cv=cv, scoring='roc_auc')

print(f"\nDecision Tree - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Decision Tree - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import classification_report, confusion_matrix

# Definir la validación cruzada estratificada
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Crear el modelo Decision Tree
decision_tree = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=10,
    max_features='sqrt',
    min_samples_leaf=1,
    min_samples_split=5,
    class_weight='balanced',
    random_state=42
)

# Obtener predicciones con validación cruzada
y2_pred = cross_val_predict(decision_tree, X2, y2, cv=cv)
```

```

# Matriz de confusión
print("\nMatriz de Confusión - Decision Tree:")
print(confusion_matrix(y2, y2_pred))

# Reporte de clasificación
print("\nReporte de Clasificación - Decision Tree:")
print(classification_report(y2, y2_pred, zero_division=0))

# Evaluación con métricas por validación cruzada
acc_cv = cross_val_score(decision_tree, X2, y2, cv=cv, scoring='accuracy')
auc_cv = cross_val_score(decision_tree, X2, y2, cv=cv, scoring='roc_auc')

print(f"\nDecision Tree - Accuracy promedio (10-CV): {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Decision Tree - AUC promedio (10-CV): {auc_cv.mean()} ± {auc_cv.std()}")

```

Estos son los resultados del modelo de árboles de decisión únicamente el parámetro `class_weight='balanced'` sin ajustar el resto de hiperparámetros, y posteriormente, combinando esta configuración con una optimización completa mediante ajuste de hiperparámetros.

Matriz de Confusión - Decision Tree:

```
[[189293  29041]
 [ 24281  11065]]
```

Reporte de Clasificación - Decision Tree:

	precision	recall	f1-score	support
0	0.89	0.87	0.88	218334
1	0.28	0.31	0.29	35346
accuracy			0.79	253680
macro avg	0.58	0.59	0.58	253680
weighted avg	0.80	0.79	0.80	253680

Decision Tree - Accuracy promedio (10-CV): 0.78980605487228 ± 0.0030726063804592542
 Decision Tree - AUC promedio (10-CV): 0.5896669208918721 ± 0.005917073919990581

Matriz de Confusión - Decision Tree:

```
[[152754  65580]
 [  8243  27103]]
```

Reporte de Clasificación - Decision Tree:

	precision	recall	f1-score	support
0	0.95	0.70	0.81	218334
1	0.29	0.77	0.42	35346
accuracy			0.71	253680
macro avg	0.62	0.73	0.61	253680
weighted avg	0.86	0.71	0.75	253680

Decision Tree - Accuracy promedio (10-CV): 0.7089916430148218 ± 0.01143852017510548
 Decision Tree - AUC promedio (10-CV): 0.8042796565860029 ± 0.00519792240598327

Igual que pasó en el caso del clasificador anterior, el uso del parámetro `class_weight` no aporta resultados nuevos. De este modo, la clara mejora que muestra el clasificador es debido al ajuste de hiperparámetros ya que realmente potencia su rendimiento.

3.4.4. Comparación de clasificadores medios tras class_weight

La siguiente es una tabla con los resultados para los modelos con class_weight='balanced', tanto sin ajuste como con ajuste de hiperparámetros, para los tres clasificadores compatibles que acaban de ser estudiados.

	Accuracy (10-CV)	AUC (10-CV)	Precision		Recall		F1-Score	
			Clase 0	Clase 1	Clase 0	Clase 1	Clase 0	Clase 1
Logistic Regression	0,8634	0,8204	0,88	0,53	0,98	0,15	0,92	0,24
Logistic Regression (con class_weight)	0,7312	0,8210	0,95	0,31	0,73	0,76	0,82	0,44
Logistic Regression (class_weight y ajustado)	0,7312	0,8210	0,95	0,31	0,73	0,76	0,82	0,44
Random Forest	0,8497	0,7472	0,88	0,41	0,96	0,19	0,92	0,26
Random Forest (con class_weight)	0,8459	0,7772	0,88	0,39	0,95	0,20	0,91	0,26
Random Forest (class_weight y ajustado)	0,7349	0,8264	0,95	0,31	0,73	0,77	0,83	0,45
Decision Tree	0,8035	0,5979	0,89	0,30	0,88	0,31	0,89	0,30
Decision Tree (con class_weight)	0,7898	0,5897	0,89	0,28	0,87	0,31	0,88	0,29
Decision Tree (class_weight y ajustado)	0,7090	0,8043	0,95	0,29	0,70	0,77	0,81	0,42

En Logistic Regression, la introducción de class_weight='balanced' mejora sustancialmente el recall de la clase minoritaria mientras que en los demás clasificadores esta mejora se debe al ajuste de hiperparámetros.

Además, en todos los modelos hay una caída en la precisión y en el accuracy del modelo. Esto es esperable, ya que el modelo comienza a etiquetar más ejemplos como positivos para capturar más verdaderos positivos, a costa de introducir más falsos positivos.

Aunque el accuracy se ve afectado, el uso conjunto de class_weight y ajuste de hiperparámetros mejora consistentemente el AUC, que mide la capacidad discriminativa del modelo.

En conclusión, el uso de clasificadores sensibles al coste permite mejorar la sensibilidad hacia la clase minoritaria, algo importante en problemas donde es crítico identificar correctamente los casos positivos, aunque esto implique sacrificar algo de precisión o accuracy general.

3.5. Tests estadísticos por pares

Para realizar una comparación objetiva entre los diferentes clasificadores evaluados, se ha aplicado una metodología estadística adecuada a escenarios donde se contrastan más de dos modelos. Esta metodología permite determinar si las diferencias observadas en el rendimiento son estadísticamente significativas y no producto del azar.

Se ha utilizado el **test de Friedman**, un test no paramétrico diseñado específicamente para comparar múltiples algoritmos evaluados sobre los mismos conjuntos de datos. Este test evalúa si existen diferencias globales significativas en el rendimiento de modelos. (Ref 30)

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.naive_bayes import GaussianNB
from scipy.stats import friedmanchisquare
import numpy as np

# Definir los modelos
logreg = LogisticRegression(max_iter=1000, random_state=42)
rf_model = RandomForestClassifier(random_state=42)
decision_tree = DecisionTreeClassifier(random_state=42)
knn = KNeighborsClassifier()
gb_model = GradientBoostingClassifier(random_state=42)
xgb_model = XGBClassifier(random_state=42)
nb_model = GaussianNB()

# Suponiendo que tus modelos están entrenados correctamente
y2 = y2.values.ravel()
auc_logreg = cross_val_score(logreg, X2, y2, cv=10, scoring='roc_auc')
auc_rf = cross_val_score(rf_model, X2, y2, cv=10, scoring='roc_auc')
auc_dt = cross_val_score(decision_tree, X2, y2, cv=10, scoring='roc_auc')
auc_knn = cross_val_score(knn, X2, y2, cv=10, scoring='roc_auc')
auc_gb = cross_val_score(gb_model, X2, y2, cv=10, scoring='roc_auc')
auc_xgb = cross_val_score(xgb_model, X2, y2, cv=10, scoring='roc_auc')
auc_nb = cross_val_score(nb_model, X2, y2, cv=10, scoring='roc_auc')

# Aplicar el test de Friedman
stat, p = friedmanchisquare(auc_logreg, auc_rf, auc_dt, auc_knn, auc_gb, auc_xgb, auc_nb)
print(f"Friedman Test - Estadístico: {stat}")
print(f"Friedman Test - p-valor: {p}")

if p < 0.05:
    print("Rechazamos la hipótesis nula: hay diferencias significativas entre los modelos.")
else:
    print("No se rechaza la hipótesis nula: no hay diferencias estadísticamente significativas.")
```

Friedman Test - Estadístico: 59.6142857142857

Friedman Test - p-valor: 5.390791555832729e-11

Rechazamos la hipótesis nula: hay diferencias significativas entre los modelos.

Para interpretar los resultados se consulta la referencia 31. El test se ha aplicado sobre las puntuaciones medias del AUC obtenidas en validación cruzada para cada modelo. El resultado del test ha indicado un rechazo de la hipótesis nula ($p < 0.05$), lo cual sugiere que existen diferencias estadísticamente significativas entre al menos algunos de los modelos comparados.

Dado que el test de Friedman ha identificado diferencias significativas, se ha aplicado el **test post-hoc de Nemenyi**, que permite realizar comparaciones por pares entre modelos. Este test compara la diferencia de rankings medios entre cada par de modelos con una diferencia crítica (CD, Critical Difference). (Ref 32)


```
!pip install scikit-posthocs

Mostrar salida oculta

import scikit_posthocs as sp
import numpy as np
import pandas as pd

# Unir las AUCs en una matriz (filas: folds, columnas: modelos)
auc_data = np.vstack([
    auc_logreg,
    auc_rf,
    auc_dt,
    auc_knn,
    auc_gb,
    auc_xgb,
    auc_nb
]).T # Transponemos para que cada fila sea un fold

# Crear DataFrame para claridad (opcional pero útil)
df_auc = pd.DataFrame(auc_data, columns=[
    'LogReg', 'RandomForest', 'DecisionTree', 'KNN', 'GradientBoosting', 'XGBoost', 'NaiveBayes'
])

# Aplicar test de Nemenyi
nemenyi_result = sp.posthoc_nemenyi_friedman(df_auc)

# Mostrar matriz de p-valores
print(nemenyi_result)
```

	LogReg	RandomForest	DecisionTree	KNN	\
LogReg	1.000000	0.436086	6.871449e-04	0.031322	
RandomForest	0.436086	1.000000	3.098440e-01	0.916230	
DecisionTree	0.000687	0.309844	1.000000e+00	0.945976	
KNN	0.031322	0.916230	9.459762e-01	1.000000	
GradientBoosting	0.370538	0.001063	1.107141e-08	0.000005	
XGBoost	0.945976	0.042787	4.725816e-06	0.000687	
NaiveBayes	0.916230	0.982115	4.278738e-02	0.436086	

	GradientBoosting	XGBoost	NaiveBayes
LogReg	3.705382e-01	0.945976	0.916230
RandomForest	1.063015e-03	0.042787	0.982115
DecisionTree	1.107141e-08	0.000005	0.042787
KNN	4.725816e-06	0.000687	0.436086
GradientBoosting	1.000000e+00	0.945976	0.022624
XGBoost	9.459762e-01	1.000000	0.309844
NaiveBayes	2.262375e-02	0.309844	1.000000

En la tabla anterior se recogen los valores p de cada comparación. Aquellos pares cuyo valor p es inferior a 0,05 indican una diferencia estadísticamente significativa entre los modelos.

El test de Nemenyi refuerza las conclusiones anteriores: los modelos de boosting (Gradient Boosting y XGBoost) ofrecen rendimientos significativamente superiores respecto a otros clasificadores más sencillos. Además, se confirma que Decision Tree es estadísticamente inferior a la mayoría, y que algunos modelos como Logistic Regression y Naive Bayes tienen un rendimiento intermedio sin diferencias claras entre ellos.

Como también se han estudiado los mismos clasificadores, pero con su debido ajuste de hiperparámetros, se considera interesante volver a realizar el test estadístico por pares.

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.naive_bayes import GaussianNB
from scipy.stats import friedmanchisquare
import numpy as np

# Clasificadores con hiperparámetros óptimos (nombres renombrados con sufijo "_h")

logreg_h = LogisticRegression(
    C=0.005, max_iter=500, penalty='l2', solver='saga', tol=1e-6, random_state=42
)

rf_model_h = RandomForestClassifier(
    bootstrap=True, max_depth=10, max_features='log2',
    min_samples_leaf=4, min_samples_split=15,
    n_estimators=200, random_state=42
)

decision_tree_h = DecisionTreeClassifier(
    criterion='entropy', max_depth=10, max_features='sqrt',
    min_samples_leaf=1, min_samples_split=5,
    random_state=42
)

knn_h = KNeighborsClassifier(
    metric='minkowski', n_neighbors=9, p=1, weights='uniform'
)

gb_model_h = GradientBoostingClassifier(
    learning_rate=0.1, max_depth=5, max_features='sqrt',
    min_samples_leaf=2, min_samples_split=5,
    n_estimators=100, subsample=0.8,
    random_state=42
)

xgb_model_h = XGBClassifier(
    colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
    min_child_weight=2, n_estimators=100, subsample=0.8,
    random_state=42
)

nb_model_h = GaussianNB(
    var_smoothing=1e-07
)

# Suponiendo que tus modelos están entrenados correctamente
y2 = y2.values.ravel()
auc_logreg_h = cross_val_score(logreg_h, X2, y2, cv=10, scoring='roc_auc')
auc_rf_h = cross_val_score(rf_model_h, X2, y2, cv=10, scoring='roc_auc')
auc_dt_h = cross_val_score(decision_tree_h, X2, y2, cv=10, scoring='roc_auc')
auc_knn_h = cross_val_score(knn_h, X2, y2, cv=10, scoring='roc_auc')
auc_gb_h = cross_val_score(gb_model_h, X2, y2, cv=10, scoring='roc_auc')
auc_xgb_h = cross_val_score(xgb_model_h, X2, y2, cv=10, scoring='roc_auc')
auc_nb_h = cross_val_score(nb_model_h, X2, y2, cv=10, scoring='roc_auc')

# Aplicar el test de Friedman
stat, p = friedmanchisquare(auc_logreg_h, auc_rf_h, auc_dt_h, auc_knn_h, auc_gb_h, auc_xgb_h, auc_nb_h)
print(f"Friedman Test con hiperparámetros - Estadístico: {stat}")
print(f"Friedman Test con hiperparámetros - p-valor: {p}")

if p < 0.05:
    print("Rechazamos la hipótesis nula: hay diferencias significativas entre los modelos.")
else:
    print("No se rechaza la hipótesis nula: no hay diferencias estadísticamente significativas.")
```

Friedman Test con hiperparámetros - Estadístico: 59.10000000000002
 Friedman Test con hiperparámetros - p-valor: 6.855759514073935e-11
 Rechazamos la hipótesis nula: hay diferencias significativas entre los modelos.

Como se encuentran diferencias significativas se vuelve a aplicar el test de Nemenyi.

```

import scikit_posthocs as sp
import numpy as np
import pandas as pd

# Unir las AUCs en una matriz (filas: folds, columnas: modelos con hiperparámetros)
auc_data_h = np.vstack([
    auc_logreg_h,
    auc_rf_h,
    auc_dt_h,
    auc_knn_h,
    auc_gb_h,
    auc_xgb_h,
    auc_nb_h
]).T # Transponemos para que cada fila sea un fold

# Crear DataFrame para claridad
df_auc_h = pd.DataFrame(auc_data_h, columns=[
    'LogReg_h', 'RandomForest_h', 'DecisionTree_h', 'KNN_h',
    'GradientBoosting_h', 'XGBoost_h', 'NaiveBayes_h'
])

# Aplicar test de Nemenyi
nemenyi_result_h = sp.posthoc_nemenyi_friedman(df_auc_h)

# Mostrar matriz de p-valores
print(nemenyi_result_h)

```

	LogReg_h	RandomForest_h	DecisionTree_h	KNN_h \
LogReg_h	1.000000	0.945976	0.945976	3.132204e-02
RandomForest_h	0.945976	1.000000	0.370538	6.871449e-04
DecisionTree_h	0.945976	0.370538	1.000000	3.705382e-01
KNN_h	0.031322	0.000687	0.370538	1.000000e+00
GradientBoosting_h	0.206646	0.830311	0.011349	8.585008e-07
XGBoost_h	0.076617	0.575532	0.002456	7.612437e-08
NaiveBayes_h	0.370538	0.031322	0.945976	9.459762e-01

	GradientBoosting_h	XGBoost_h	NaiveBayes_h
LogReg_h	2.066457e-01	7.661717e-02	0.370538
RandomForest_h	8.303106e-01	5.755324e-01	0.031322
DecisionTree_h	1.134911e-02	2.456375e-03	0.945976
KNN_h	8.585008e-07	7.612437e-08	0.945976
GradientBoosting_h	1.000000e+00	9.996147e-01	0.000173
XGBoost_h	9.996147e-01	1.000000e+00	0.000024
NaiveBayes_h	1.732981e-04	2.362085e-05	1.000000

El test confirma que los modelos de Gradient Boosting y XGBoost siguen siendo los de mejor desempeño incluso tras el ajuste de hiperparámetros, con diferencias estadísticamente significativas respecto a varios modelos más simples. Modelos como KNN y Decision Tree quedan rezagados, mientras que Random Forest y Logistic Regression ofrecen un rendimiento intermedio sin diferencias claras respecto a los mejores.

3.6. Cálculos adicionales

Tal como se mencionó al finalizar el análisis del balanceo de clases en el nivel básico, la combinación de **RandomForestClassifier sin ajuste de hiperparámetros junto con SMOTE** resultó ser la más efectiva hasta el momento, alcanzando un AUC de **0,8876**, el más alto registrado en todo el estudio.

Sin embargo, este resultado se obtuvo trabajando sobre la tabla X, en la que se habían eliminado algunas variables que más adelante se descubrió eran relevantes. Por eso, se considera necesario repetir esta misma configuración (clasificador, balanceo y parámetros), pero aplicándola ahora sobre la tabla **X2**, que conserva esas variables importantes. El objetivo es comprobar si es posible mejorar aún más el AUC alcanzado previamente.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import cross_val_score
from imblearn.over_sampling import SMOTE

# 1. SMOTE sobre el conjunto de entrenamiento
smote = SMOTE(random_state=42)
X2_train_smote, y2_train_smote = smote.fit_resample(X2_train, y2_train)
y2_train_smote = y2_train_smote.values.ravel()

# 2. Entrenar el modelo
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X2_train_smote, y2_train_smote)

# 3. Predecir sobre el conjunto de test original
y2_pred = rf_model.predict(X2_test)

# 4. Matriz de confusión
conf_mat = confusion_matrix(y2_test, y2_pred)
print("Matriz de Confusión:\n", conf_mat)

# 5. Reporte de clasificación
report = classification_report(y2_test, y2_pred, zero_division=0)
print("\nReporte de Clasificación:\n", report)

# 6. 10-CV para Accuracy
acc_cv = cross_val_score(rf_model, X2_train_smote, y2_train_smote, cv=10, scoring='accuracy')

# 7. 10-CV para AUC
auc_cv = cross_val_score(rf_model, X2_train_smote, y2_train_smote, cv=10, scoring='roc_auc')

print(f"\nRandom Forest - Accuracy promedio (10-CV) con SMOTE: {acc_cv.mean()} ± {acc_cv.std()}")
print(f"Random Forest - AUC promedio (10-CV) con SMOTE: {auc_cv.mean()} ± {auc_cv.std()}")
```

Matriz de Confusión:

```
[[53790 11815]
 [ 5340  5159]]
```

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.91	0.82	0.86	65605
1	0.30	0.49	0.38	10499
accuracy			0.77	76104
macro avg	0.61	0.66	0.62	76104
weighted avg	0.83	0.77	0.80	76104

Random Forest - Accuracy promedio (10-CV) con SMOTE: 0.8631400115004105 ± 0.02282466669818411
Random Forest - AUC promedio (10-CV) con SMOTE: 0.9422190831865749 ± 0.019928576943387392

Con la combinación de Random Forest con SMOTE (sin ajuste de hiperparámetros) estudiado sobre la tabla X2 se obtiene un **AUC promedio de 0,9422**. Este es el valor más alto obtenido hasta ahora, superando ampliamente a todas las configuraciones previas. Esto indica que el modelo ha mejorado notablemente su capacidad para discriminar entre clases.

Por ello, se puede concluir que la combinación de Random Forest sin ajuste y SMOTE proporciona el mejor AUC entre todos los modelos evaluados, lo que la convierte en una estrategia muy eficaz para manejar el desbalance de clases, especialmente si el objetivo es mejorar la capacidad de detección de la clase minoritaria sin perder demasiada estabilidad en el rendimiento global.

IMPORTANTE: la combinación de RandomForestClassifier sin ajuste y SMOTE trabajando sobre la tabla X2 ofrece el valor más alto de área bajo la curva ROC, con un promedio de **0,9422**.

4. Referencias

1. W3Schools.com. (2025). W3schools.com. https://www.w3schools.com/python/matplotlib_histograms.asp
2. *seaborn.heatmap* — *seaborn 0.13.2 documentation*. (2024). Pydata.org. <https://seaborn.pydata.org/generated/seaborn.heatmap.html>
3. *A Guide to Scikit-Learn's Dummy Classifiers - Sling Academy*. (2024). Slingacademy.com. <https://www.slingacademy.com/article/a-guide-to-scikit-learn-s-dummy-classifiers/>
4. *Cross-validation: evaluating estimator performance*. (2025). Scikit-Learn. https://scikit-learn.org/stable/modules/cross_validation.html
5. *confusion_matrix*. (2025). Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html
6. *classification_report*. (2025). Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html
7. OpenAI. (2025). *ChatGPT* (versión GPT-4) [Modelo de lenguaje]. <https://chat.openai.com/>
8. *Logistic Regression with Cross-Validation in Scikit-Learn - Sling Academy*. (2024). Slingacademy.com. <https://www.slingacademy.com/article/logistic-regression-with-cross-validation-in-scikit-learn/>
9. *cross_val_predict*. (2025). Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.cross_val_predict.html?utm_source=chatgpt.com
10. Shafi, A. (2018, May 16). *Random Forest Classification with Scikit-Learn*. Datacamp.com; DataCamp. <https://www.datacamp.com/tutorial/random-forests-classifier-python>
11. *RandomizedSearchCV*. (2025). Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html
12. *LogisticRegression*. (2025). Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
13. *Python Examples of sklearn.model_selection.RandomizedSearchCV*. (2025). Programcreek.com. https://www.programcreek.com/python/example/91146/sklearn.model_selection.RandomizedSearchCV
14. *RandomForestClassifier*. (2025). Scikit-Learn. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

15. in. (2019, April 9). *How to perform SMOTE with cross validation in sklearn in python*. Stack Overflow. <https://stackoverflow.com/questions/55591063/how-to-perform-smote-with-cross-validation-in-sklearn-in-python>
16. *RandomUnderSampler* — Version 0.13.0. (2024). Imbalanced-Learn.org. https://imbalanced-learn.org/stable/references/generated/imblearn.under_sampling.RandomUnderSampler.html
17. *wilcoxon* — SciPy v1.15.2 Manual. (2025). Scipy.org. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.wilcoxon.html>
18. Bobbitt, Z. (2020, July 10). *How to Conduct a Wilcoxon Signed-Rank Test in Python*. Statology. <https://www.statology.org/wilcoxon-signed-rank-test-python/>
19. *SelectKBest*. (2025). Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectKBest.html
20. *RFE*. (2025). Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.RFE.html
21. Nik. (2022, February 9). *Hyper-parameter Tuning with GridSearchCV in Sklearn • datagy*. Datagy. <https://datagy.io/sklearn-gridsearchcv/>
22. *How to Implement Grid Search Using GridSearchCV in Python*. (2019, September 19). Datatechnotes.com. https://www.datatechnotes.com/2019/09/how-to-use-gridsearchcv-in-python.html#google_vignette
23. *DecisionTreeClassifier*. (2025). Scikit-Learn. <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
24. *KNeighborsClassifier*. (2025). Scikit-Learn. <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>
25. *GradientBoostingClassifier*. (2025). Scikit-Learn. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html>
26. *XGBoost Parameters* — *xgboost 3.1.0-dev documentation*. (2025). Readthedocs.io. <https://xgboost.readthedocs.io/en/latest/parameter.html#global-configuration>
27. 1.9. *Naive Bayes*. (2025). Scikit-Learn. https://scikit-learn.org/stable/modules/naive_bayes.html
28. *compute_class_weight*. (2025). Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.utils.class_weight.compute_class_weight.html
29. GeeksforGeeks. (2024, August 7). *How Does the class_weight Parameter in ScikitLearn Work?* GeeksforGeeks. <https://www.geeksforgeeks.org/how-does-the-classweight-parameter-in-scikit-learn-work/>
30. *friedmanchisquare* — SciPy v1.15.2 Manual. (2025). Scipy.org. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.friedmanchisquare.html>

31. Bobbitt, Z. (2020, July 13). *How to Perform the Friedman Test in Python*. Statology. <https://www.statology.org/friedman-test-python/>
32. GeeksforGeeks. (2022, February 26). *How to Perform the Nemenyi Test in Python*. GeeksforGeeks. <https://www.geeksforgeeks.org/how-to-perform-the-nemenyi-test-in-python/>

5. Bibliografía

Diabetes. (2024). Medlineplus.gov; National Library of Medicine. <https://medlineplus.gov/spanish/diabetes.html#:~:text=%C2%BFQu%C3%A9%20es%20la%20diabetes%3Fde%20los%20alimentos%20que%20consume>.

Ministerio de Sanidad, Consumo y Bienestar Social - Ciudadanos - La Diabetes. (2025). Sanidad.gob.es. <https://www.sanidad.gob.es/ciudadanos/enfLesiones/enfNoTransmisibles/diabetes/diabetes.htm>

CDC. (2022, August 29). *Para adultos*. Centers for Disease Control and Prevention. https://www.cdc.gov/healthyweight/spanish/assessing/bmi/adult_bmi/index.html

Diabetes e hipertensión arterial, ¿hay relación? | SegurCaixa Adeslas. (2021, July 30). Segurcaixaadeslas.es. <https://www.segurcaixaadeslas.es/espacio-de-salud-y-bienestar/diabetes-e-hipertension-arterial-hay-relacion>

La conexión entre la diabetes, la insuficiencia renal y la alta presión arterial. (2020, November 3). Wwww.heart.org. <https://www.heart.org/en/news/2020/11/03/la-conexion-entre-la-diabetes-la-insuficiencia-renal-y-la-alta-presion-arterial>

Colesterol y diabetes. (2018, January 29). Wwww.goredforwomen.org. <https://www.goredforwomen.org/es/health-topics/diabetes/diabetes-complications-and-risks/cholesterol-abnormalities--diabetes>

Excess Weight and Type 2 Diabetes. (2025). Translate.goog. https://www-honorhealth-com.translate.goog/medical-services/bariatric-weight-loss-surgery/patient-education-and-support/comorbidities-type-2-diabetes?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=rq

CDCespanol. (2022, May 19). *El tabaquismo y la diabetes*. Centers for Disease Control and Prevention. <https://www.cdc.gov/tobacco/campaign/tips/spanish/enfermedades/tabaquismo-diabetes.html>

Problemas del corazón asociados con la diabetes. (2024). Medlineplus.gov; National Library of Medicine. <https://medlineplus.gov/spanish/diabeticheartdisease.html>

Hacer ejercicio sirve para prevenir la diabetes y reducir la mortalidad cardiovascular | Hospit. (2020, November 12). Clínic Barcelona. <https://www.clinicbarcelona.org/noticias/hacer-ejercicio-sirve-para-prevenir-la-diabetes-y-reducir-la-mortalidad-cardiovascular>

Reuters. (2012, April 27). *Comer frutas y vegetales reduce el riesgo de diabetes: estudio*. Reuters. <https://www.reuters.com/article/world/comer-frutas-y-vegetales-reduce-el-riesgo-de-diabetes-estudio-idUSSIE83Q0AK/>

Hermida, Á. (2020, June 20). *Comer más verduras reduce el riesgo de diabetes un 60%*. Alimento.elconfidencial.com; El Confidencial. https://www.alimento.elconfidencial.com/nutricion/2020-06-20/diabetes-tipo-2-frutas-y-verduras-enfermedades_2068474/

West, M. (2021, December 31). *Lo que debes saber sobre la diabetes tipo 2 y el alcohol*. Medicalnewstoday.com; Medical News Today. <https://www.medicalnewstoday.com/articles/es/diabetes-tipo-2-y-alcohol>

Una complicación de la diabetes que, por lo general, puede prevenirse-Neuropatía diabética - Síntomas y causas - Mayo Clinic. (2022). Mayo Clinic. <https://www.mayoclinic.org/es/diseases-conditions/diabetic-neuropathy/symptoms-causes/syc-20371580>